

KEMENTERIAN PENDIDIKAN,
KEBUDAYAAN, RISET, DAN
TEKNOLOGI
REPUBLIK INDONESIA, 2021

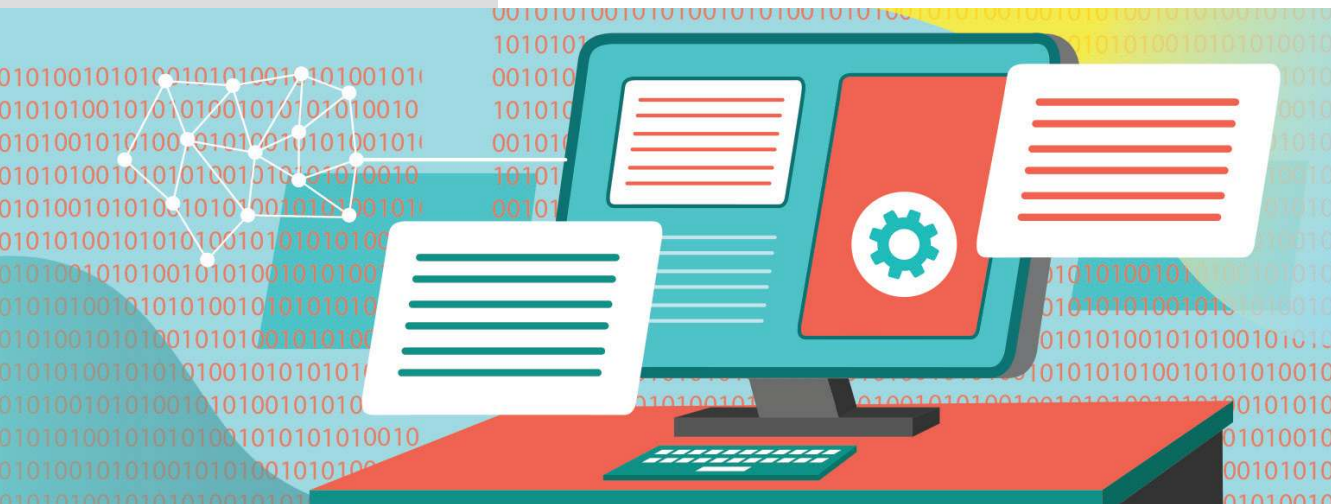
Informatika untuk SMA Kelas XI

Penulis : Mushtofa
Auzi Asfarian
Dean Apriana Ramadhan

ISBN: 978-602-244-861-7

2

Strategi Algoritmik dan Pemrograman



Tujuan Pembelajaran

Setelah mempelajari bab ini kalian diharapkan mampu menganalisis beberapa strategi algoritmik secara kritis untuk menyelesaikan suatu permasalahan dengan memberikan justifikasi efisiensi. Setelah itu, kalian akan menerapkan strategi algoritmik tersebut ke dalam suatu program yang lebih kompleks dari program yang telah kalian kerjakan sewaktu Kelas X. Kalian akan belajar bergotong royong dalam mengembangkan dan menguji program tersebut untuk menyelesaikan problem yang berkaitan dengan mata pelajaran lain yang telah kalian pelajari.

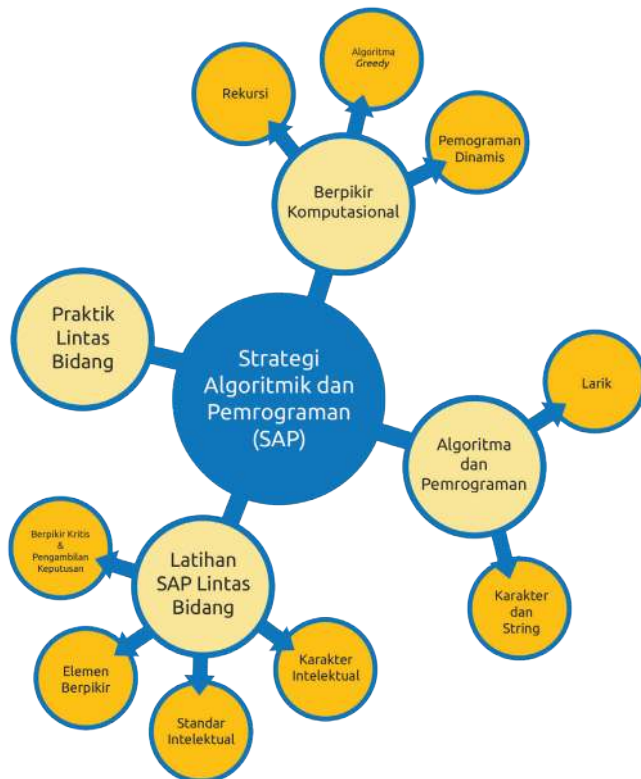


Pertanyaan Pemantik

Pernahkah kalian berpikir bagaimana program yang kita gunakan dalam kehidupan sehari-hari dikembangkan? Bagaimana para pemrogram menghasilkan program yang sangat kompleks yang kalian gunakan di kehidupan sehari-hari? Bagaimana program tersebut dirancang dan diimplementasikan hingga dapat melayani ribuan hingga jutaan pengguna dengan efisien sehingga program tetap terasa nyaman saat digunakan?



Peta Konsep



▲ Gambar 2.1 Peta Konsep Strategi Algoritmik dan Pemrograman



Apersepsi

Dalam kehidupan ini, kalian pastilah telah menggunakan berbagai program untuk mempermudah aktivitas di berbagai bidang, seperti komunikasi, kesehatan, sosial, transportasi, hingga kebutuhan pokok sehari-hari. Akan tetapi, program yang bermanfaat tidak sekedar dapat dijalankan, namun bagaimana program tersebut dapat berfungsi dengan benar dan efisien sehingga dapat digunakan oleh manusia dengan nyaman. Untuk menghasilkan program demikian, diperlukan suatu kemampuan menyusun strategi algoritmik dan pemrograman.



Kata Kunci

Strategi algoritmik, pemrograman, penyelesaian masalah.

A. Proses Pemrograman

Pada buku Informatika SMA Kelas X, kalian telah mempelajari proses menulis kode program atau yang kita sebut sebagai *coding*. Pada Kelas XI, kalian akan mempelajari kegiatan yang lebih kompleks dari *coding* yang kita sebut sebagai pemrograman. Pemrograman menurut buku “Oxford Dictionary of Computer Science” adalah seluruh aktivitas teknis yang dilakukan untuk menghasilkan suatu program, termasuk analisis kebutuhan dan seluruh langkah desain dan implementasi suatu program.

Program dapat menjadi solusi dari suatu permasalahan. Untuk menghasilkan program yang benar dan dapat membantu manusia dalam melakukan tugasnya, ada empat langkah yang dilakukan pada saat melakukan pemrograman, yaitu:

1. **Menganalisis permasalahan (*Analyzing*):** pemrogram menganalisis suatu kebutuhan atau keadaan saat ini untuk menghasilkan definisi permasalahan yang perlu

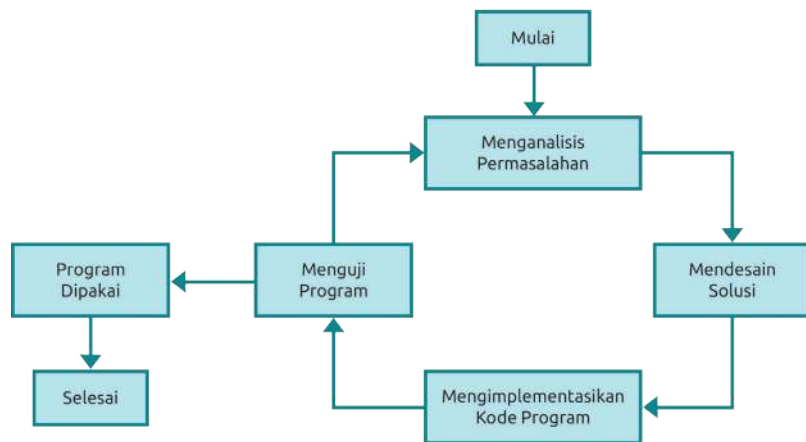
diselesaikan dengan program. Permasalahan yang dianalisis bisa berupa masalah yang baru atau penyempurnaan dari solusi yang sudah ada. Kemampuan berpikir komputasional digunakan untuk mencari abstraksi dari permasalahan yang akan diselesaikan. Permasalahan yang kompleks dapat didekomposisi ke beberapa masalah yang lebih kecil, namun saling berkaitan. Hingga akhirnya, pemrogram akan mengenali pola permasalahan tersebut sebagai sebuah variasi dari *problem* generik. Tahap ini menghasilkan pernyataan masalah (*problem statement*) yang menjelaskan masukan (*input*), keluaran (*output*), serta batasan-batasan (*constraint*) dari program yang akan dibuat.

2. **Mendesain solusi (*Problem Solving*):** dari pernyataan masalah ini, pemrogram merencanakan strategi untuk menghasilkan keluaran berdasarkan masukan yang diterima. Pemrogram tidak harus merencanakan solusi dari awal. Mereka dapat menggunakan solusi atau potongan solusi yang sudah ada dari permasalahan yang telah diselesaikan sebelumnya. Bahkan, jika problem generik telah ditemukan, pemrogram dapat memodifikasi algoritma generik agar sesuai dengan permasalahan. Proses ini akan menghasilkan algoritma berupa narasi, *pseudocode*, atau **diagram alir**. Pada tahap ini, pemrogram juga akan mengevaluasi algoritma yang dibuat untuk memenuhi batasan dari permasalahan. Misalnya, apakah program dapat bekerja dengan cepat (kurang dari 1 detik) saat diberikan masukan yang berukuran besar, karena menunggu membuat pengguna tidak nyaman.
3. **Mengimplementasikan solusi dalam bentuk program (*Coding*):** Pada tahap ini, pemrogram akan menulis kode program untuk menjalankan solusi yang telah direncanakan sebelumnya dengan menggunakan suatu bahasa pemrograman. Memilih bahasa pemrograman menjadi pertimbangan di tahap ini. Selain itu, mengubah

algoritma menjadi kode program juga melibatkan banyak pertimbangan teknis (misalnya: tipe data, struktur kontrol yang digunakan, dan lain-lain).

4. **Menguji program (*Testing*):** Setelah program dapat dijalankan, program tersebut harus diuji untuk memastikan program berjalan dengan benar, sesuai dengan batasan-batasan yang diberikan. Pengujian dapat dilakukan dengan menggunakan berbagai strategi pengujian. Salah satu yang telah kalian pelajari di Kelas X adalah menguji program menggunakan kasus uji (*test case*) yang dibuat sedemikian rupa sehingga mewakili seluruh kemungkinan masukan dari program.

Keempat tahap di atas seringkali tidak dilakukan satu kali, tapi berkali-kali sehingga membentuk suatu siklus pemrograman (lihat [Gambar 2.2](#)).

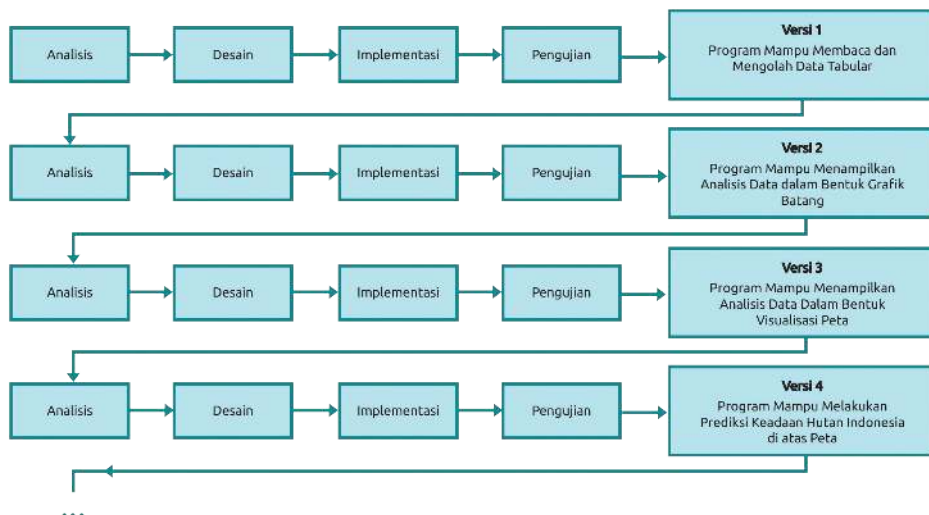


▲ **Gambar 2.2** Siklus Pemrograman

Dalam latihan di buku Informatika Kelas X, kalian mungkin merasakan bahwa program yang kalian buat masih jauh lebih sederhana dibandingkan dengan program yang biasa kalian gunakan dalam kehidupan sehari-hari. Hal tersebut sangatlah wajar, karena program-program yang kompleks dengan tampilan visual dan interaktif yang banyak dipakai saat ini telah melewati tahapan pengembangan yang panjang dan

telah melalui beberapa versi. Apabila kalian menggunakan sebuah program yang disediakan secara daring (misalnya untuk berbelanja) dan rutin dalam waktu lama, kalian akan merasakan bahwa program tersebut senantiasa mengalami perubahan dan perbaikan atau penambahan fitur secara terus menerus. Hal ini menjadi bagian dari “siklus kehidupan” suatu program agar terus relevan dengan kebutuhan penggunaanya.

Pendekatan siklus ini juga bermanfaat jika kalian ingin membuat sebuah program yang kompleks. Kalian dapat menetapkan suatu bentuk akhir dari program yang akan kalian buat dan mendekomposisinya ke dalam program-program antara yang lebih sederhana. Misalnya, kalian ingin membuat program yang mampu memprediksi kondisi hutan Indonesia di masa depan. Untuk membuat program yang kompleks tadi, kalian dapat mulai dengan membuat program yang dapat membaca dan mengolah data hutan dalam bentuk tabel terlebih dahulu, kemudian menambahkan analisis data sederhana dalam bentuk grafik batang, kemudian mengubahnya menjadi visualisasi berbentuk peta, baru akhirnya menambah kemampuan prediksi keadaan hutan (lihat [Gambar 2.3](#)).



Gambar 2.3 Contoh Pembuatan suatu Program Prediksi Keadaan Hutan Indonesia yang Dibangun secara Iteratif

► Praktik Baik Pemrograman

Kalian perlu mengumpulkan dengan rapi hasil pekerjaan dari langkah di atas. Misalnya, coret-coretan, algoritma yang dibuat, kode sumber, serta proses pengujian yang dilakukan. Hal ini disebut **dokumentasi** program. Dokumentasi berguna bagi kalian maupun orang lain untuk memahami kode program dengan cepat. Dokumentasi dapat ditulis baik di dalam kode program maupun di dokumen terpisah (untuk program yang kompleks). Dokumentasi yang ditulis menyatu dengan kode dapat ditulis menggunakan komentar, namun komentar tidak perlu menjelaskan setiap baris kode, cukup menjelaskan abstraksi/ide dari kode. Ingatlah bahwa kode program yang ditulis dengan baik sudah pasti akan mudah dipahami.

Contoh Kasus Siklus Pemrograman

Selanjutnya mari kita coba lakukan keempat langkah tersebut untuk membuat sebuah program yang dapat membantu seseorang mengidentifikasi jenis segitiga berdasarkan panjang ketiga sisinya. Seperti kita ketahui, ada beberapa jenis segitiga berdasarkan kondisi panjang masing-masing sisinya, misalnya segitiga sama sisi, segitiga sama kaki, atau pun segitiga sembarang. Bayangkan sebuah program yang dapat menentukan apakah sebuah segitiga termasuk segitiga sama kaki, sama sisi, atau bukan keduanya, jika diberikan data berupa ketiga panjang sisi-sisi dari segitiga yang dimaksud. Misalnya jika diberikan masukan berupa panjang sisi-sisi: 2, 2 dan 3, maka program tersebut tentunya harus menghasilkan keluaran “segitiga sama kaki”.

1. Mendefinisikan Permasalahan

Deskripsi di atas sangat abstrak dan luas. Oleh karena itu, sebagai pemrogram kita perlu mendefinisikan ruang lingkup dari permasalahan yang diberikan. Pada pengembangan perangkat lunak, ruang lingkup tersebut disebut sebagai kebutuhan (requirement). Definisi ini perlu dibuat dengan baik karena pengecekan kebenaran program yang kita buat akan bergantung pada definisi permasalahan yang telah dibuat. Untuk menggali

requirement, kalian dapat bertanya kepada pembuat soal, kepada guru, atau kepada orang-orang yang nantinya akan menggunakan program kalian.

Misalnya pada deskripsi mengidentifikasi jenis segitiga di atas, kita perlu mencari jawaban dari beberapa pertanyaan berikut: (a) Bagaimana caranya mengidentifikasi apakah data panjang sisi-sisi tersebut menunjukkan sisi sebuah segitiga? (karena bisa jadi tidak ada segitiga yang memiliki panjang sisi-sisi sebagaimana data yang diberikan) (b) Bagaimana menentukan jenis segitiga dari panjang sisi-sisinya? (c) Jenis segitiga apa saja yang harus kita identifikasi (misalnya: apakah kita harus bisa mengidentifikasi segitiga siku-siku?) (d) Batasan seperti apa yang harus dipenuhi oleh data masukan? (e) keluaran seperti apa yang harus diberikan oleh program? dan seterusnya. Setelah pertanyaan-pertanyaan tersebut terjawab, kalian dapat membuat definisi permasalahan yang lebih formal. Contoh berikut dapat menjadi salah satu cara untuk mendefinisikan permasalahan ini setelah kalian mendapatkan jawaban dari pertanyaan-pertanyaan di atas.

Mengidentifikasi Jenis Segitiga

Deskripsi:

Diberikan tiga buah bilangan bulat yang berada pada rentang $[1, 1000]$ yang merupakan panjang sisi dari sebuah segitiga. Identifikasi apakah ketiga sisi tersebut membentuk segitiga sama sisi, atau segitiga sama kaki, segitiga sembarang, atau tidak bisa membentuk segitiga!

Masukan:

Masukan terdiri atas tiga bilangan bulat a , b , dan c yang merupakan panjang masing-masing sisi segitiga pada rentang $[1, 1000]$

Proses:

Dari masukan yang diberikan, kita harus menentukan terlebih dahulu, apakah ada segitiga dengan panjang sisi-sisi a , b dan c . Jika tidak ada, maka program dapat berhenti dan melaporkan

bahwa data masukan tidak menunjukkan sisi-sisi sebuah segitiga. Jika ternyata a, b dan c merupakan sisi-sisi sebuah segitiga, maka program harus menentukan jenis segitiga apakah yang memiliki panjang sisi-sisi a, b dan c tersebut.

Keluaran:

Keluaran berupa teks sebagai berikut ini:

- “Segitiga Sama Sisi” jika masukan berupa segitiga sama sisi.
- “Segitiga Sama Kaki” jika masukan berupa segitiga sama kaki.
- “Segitiga Sembarang” jika masukan berupa segitiga sembarang.
- “Bukan Segitiga” jika masukan bukan berupa segitiga.

2. Merancang Solusi

Setelah definisi persoalan (masukan, proses, keluaran) dibuat seperti di atas. Selanjutnya kalian akan mencoba menyusun algoritma untuk menyelesaikan permasalahan tersebut. Pertama-tama, untuk menentukan apakah ada segitiga yang memiliki sisi-sisi dengan panjang sesuai masukan, yaitu a, b dan c, maka nilai-nilai tersebut haruslah memenuhi aturan “Teorema Pertidaksamaan Segitiga” berikut:

“Untuk semua segitiga dengan panjang a, b dan c, maka haruslah berlaku $a + b > c$ ”

Jika kita terjemahkan teorema di atas untuk sebarang masukan a, b, dan c, kita harus memeriksa 3 buah kondisi berikut:

- $a + b > c$
- $a + c > b$
- $b + c > a$

Ketiga kondisi tersebut harus terpenuhi, agar a, b dan c dapat membentuk segitiga. Dengan kata lain, jika setidaknya salah satu kondisi tersebut tidak dipenuhi, maka ketiga sisi tersebut tidak bisa membentuk segitiga. Jika hal ini terjadi, program dapat melaporkan hasil ini dan kemudian langsung berhenti.

Jika tidak, berarti a , b dan c memang benar merupakan sisi-sisi sebuah segitiga, dan program dapat mengidentifikasi jenis segitiga yang sesuai.

Selanjutnya, untuk mengidentifikasi jenis segitiga, kita dapat merancang aturan-aturan sebagai berikut:

- Jika tiga panjang sisi sama, maka segitiga tersebut adalah sama sisi;
- Jika hanya dua panjang sisi sama, maka segitiga tersebut sama kaki;
- Jika ketiga panjang sisi berbeda, maka segitiga tersebut adalah segitiga sembarang.

3. Mengimplementasikan Solusi dalam Bentuk Program (Coding)

Selanjutnya, kalian dapat mengimplementasikan algoritma tersebut ke dalam suatu program komputer. Misalnya, kita dapat mencoba implementasikan dalam bahasa pemrograman C. Tentunya, kalian juga dapat menuliskan algoritma di atas dalam bahasa pemrograman lainnya yang telah kalian pelajari seperti Python, atau bahkan menggunakan pemrograman blok.

Catatan: Saat menuliskan kode yang ada pada buku ini, kalian tidak perlu menuliskan nomor baris pada program kalian. Nomor baris pada buku ini ditambahkan untuk mempermudah kalian dalam membaca kode sumber.

```

1:  /* Program untuk Menentukan Jenis Segitiga */
2:  #include <stdio.h>
3:  int main()
4:  {
5:      int a, b, c;
6:      scanf("%d %d %d", &a, &b, &c);
7:
8:      if ((a >= b + c) || (b >= a + c) || (c >= a + b))
9:          printf("Bukan Segitiga");
10:     else
11:         if ((a == b) && (b == c))
12:             printf("Segitiga Sama Sisi");
13:         else if ((a == b) || (b == c) || (a == c))
14:             printf("Segitiga Sama Kaki");
15:         else
16:             printf("Segitiga Sembarang");
17:     return 0;
18: }

```

4. Menguji Program

Setelah program selesai, kita perlu menguji kebenaran program tersebut. Ada banyak metode formal untuk menguji suatu program, namun yang akan dicontohkan adalah menguji program menggunakan kasus uji seperti yang telah diberikan di Kelas X. Misalkan, kita akan menguji kode program di atas dengan menggunakan pasangan kasus uji pada tabel berikut:

▼ Tabel 2.1 Lembar kerja untuk menguji Program Jenis Segitiga

No	Masukan	Keluaran Seharusnya	Keluaran Program	Hasil Benar?
1	5 5 5	Segitiga Sama Sisi		
2	1000 1000 500	Segitiga Sama Kaki		
3	100 200 500	Bukan Segitiga		
4	100 400 200	S e g i t i g a Sembarang		

Apabila terdapat kasus uji yang tidak dapat menghasilkan keluaran yang benar oleh program, lakukan analisis kembali

untuk mengidentifikasi kesalahan tersebut dan memperbaiki program kalian hingga benar.

► Ini Informatika

Dari ilustrasi di atas, kalian telah merasakan bahwa dalam membuat program, kalian perlu memahami permasalahan yang kalian kerjakan. Semakin kompleks suatu permasalahan yang harus diselesaikan, atau semakin banyak fitur yang harus diimplementasikan, program juga akan menjadi semakin kompleks dan mungkin menjadi panjang (lebih banyak instruksinya). Oleh karena itu, tim yang membuat program akan terdiri atas banyak orang dengan keahlian dan spesialisasi yang berbeda. Di antara mereka juga ada orang-orang yang ahli dan paham dengan permasalahan yang diselesaikan.



Ayo Berlatih!

Aktivitas Individu

Aktivitas SAP-K11-01-U: Mengamati Evolusi Program

Deskripsi Tugas

Pada bagian ini telah dijelaskan bahwa suatu program dikembangkan setahap demi setahap dalam suatu siklus. Hal ini juga berlaku pada program, baik berbentuk aplikasi atau web yang kalian gunakan. Sekarang pilihlah satu aplikasi atau web yang sering kalian gunakan, kemudian buatlah infografis linimasa (timeline) sederhana yang menunjukkan proses perubahan program seiring dengan berjalannya waktu. Kalian dapat melakukan pencarian untuk menemukan versi lama dari program. Untuk web, kalian dapat menggunakan aplikasi web seperti web.archive.org yang dapat menyimpan versi lama dari web secara periodik.



Ayo Renungkan!

Setelah selesai melakukan aktivitas tersebut. Jawablah pertanyaan berikut ini dalam Lembar Refleksi pada Buku Kerja, dan jangan lupa mencatat kegiatan dalam Jurnal Peserta Didik.

1. Apakah kalian telah dapat memahami setiap proses dalam pemrograman?
2. Tahap apa yang menurut kalian paling menarik untuk dilakukan?
3. Tahap apa yang menurut kalian paling sulit untuk dilakukan?
4. Pelajaran paling berkesan apa yang Kalian dapatkan dari latihan ini?

B. Berpikir Komputasional

Berpikir komputasional ini merupakan suatu kerangka dan proses berpikir yang mencakup perangkat keras, perangkat lunak, dan menalar (*reasoning*) mengenai sistem dan persoalan. Moda berpikir (*thinking mode*) ini didukung dan dilengkapi dengan pengetahuan teoritis dan praktis, serta teknik untuk menganalisis, memodelkan dan menyelesaikan persoalan. Berikut ini pembahasan beberapa konsep dan strategi berpikir komputasional yang biasa digunakan dalam menyelesaikan persoalan komputasi.

1. Rekursi

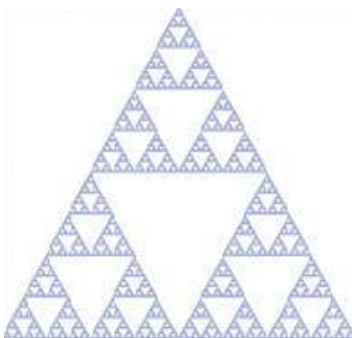
Pada bagian ini kalian akan mempelajari tentang konsep rekursi dan beberapa contoh permasalahan yang dapat diselesaikan dengan menggunakan konsep tersebut, salah satunya adalah [barisan Fibonacci](#).



▲ Gambar 2.4 Ilustrasi dari Rekursi

Suatu masalah dapat didekomposisi menjadi permasalahan yang serupa, namun ukurannya lebih kecil. Saat kita diminta untuk memindahkan satu kardus buku yang sangat berat dan tidak dapat kita angkat, kita akan membagi kardus tersebut ke dalam beberapa kardus yang lebih ringan sehingga pekerjaan tersebut menjadi lebih mudah untuk dikerjakan. Ketika menghitung suatu nilai faktorial, kita pun harus menghitung nilai faktorial yang lebih kecil. Misalnya, ketika menghitung 10 faktorial, kita juga harus menyelesaikan 1 faktorial, 2 faktorial, hingga 9 faktorial terlebih dahulu.

Secara alami, terdapat banyak permasalahan yang dapat dimodelkan dengan lebih mudah menggunakan konsep rekursif ini. Pada bagian ini, kalian akan mempelajari konsep dasar rekursi yang akan sangat berguna untuk melakukan dekomposisi pada suatu permasalahan besar dalam bentuk permasalahan yang lebih kecil dan lebih mudah untuk diselesaikan. Rekursi didefinisikan sebagai “sesuatu” yang mengandung “sesuatu” itu sendiri. Dapatkah kalian melihat rekursi dalam gambar-gambar sebagai berikut (lihat [Gambar 2.5](#))



Segi tiga Sierpinski



Boneka bersarang dari Rusia

▲ **Gambar 2.5** Beberapa contoh Rekursi di Kehidupan Sehari-hari

Dalam pembahasan kali ini, kita akan membahas fungsi/barisan rekurensi (recurrence) yaitu fungsi/barisan dimana nilai dari

fungsi atau barisan tersebut ditentukan/tergantung dari nilai fungsi/barisan itu sendiri secara rekursif, pada urutan nilai-nilai sebelumnya. Misalnya, kita memiliki sebuah barisan $a_i, i=1,2,\dots,n$ sebagai berikut:

$$\{a_i\} = 1, 3, 5, 7, \dots$$

Dimana nilai pertama dari barisan (a_1) adalah 1, dan kemudian nilai-nilai berikutnya dalam barisan tersebut dihitung dengan cara menambahkan nilai 2 kepada nilai barisan sebelumnya. Kita dapat menuliskan dalam notasi rekursif sebagai berikut:

$$a_i = \begin{cases} 1, & \text{jika } i = 1 \\ a_{i-2} + 2, & \text{jika } i > 1 \end{cases}$$

Pada definisi sebuah barisan/fungsi rekursif, selalu ada minimal dua hal yang harus ditentukan, yaitu:

- **Basis:** menunjukkan dasar/nilai awal dari fungsi/barisan tersebut. Misalnya, pada contoh di atas, $a_1=1$
- **Rekursi:** menunjukkan hubungan antara nilai dari fungsi/barisan tersebut dengan nilai-nilai sebelumnya yang telah diketahui. Misalnya, pada contoh di atas: $a_i=a_{i-1}+2$, jika $i > 1$.

Sebuah fungsi/barisan rekursif bisa jadi ditentukan dari tidak hanya satu buah nilai sebelumnya saja, tetapi dapat juga dari 2, 3, ... dan seterusnya, nilai sebelumnya. Sebagai contoh sebuah barisan dapat didefinisikan sebagai berikut:

$$a_i = \begin{cases} 1, & \text{jika } i = 1 \text{ atau } i = 2 \\ a_{i-1} + a_{i-2} & \text{jika } i > 2 \end{cases}$$

Barisan ini dimulai dengan nilai 1, kemudian untuk menentukan nilai berikutnya, kita hitung dengan cara menjumlahkan dua nilai sebelumnya pada barisan tersebut, sehingga didapatkan barisan sebagai berikut:

$$\{a_i = 1, 1, 2, 3, 5, 8, 13, 21, \dots\}$$

Barisan di atas biasa disebut sebagai barisan Fibonacci, karena dipopulerkan oleh seorang matematikawan Italia bernama Fibonacci (nama lengkap **Leonardo Bonacci**, 1170 – 1250 M). Perlu diperhatikan bahwa karena pada bagian rekursi, kita memerlukan dua nilai terakhir, pada bagian basis, kita perlu mendefinisikan dua nilai pertama dari barisan tersebut. Secara umum, banyaknya nilai yang harus didefinisikan pada bagian basis ditentukan oleh banyaknya suku barisan yang diperlukan pada bagian definisi rekursi.



Ayo Berlatih!

Aktivitas Individu

Aktivitas SAP-K11-02-U: Memahami Relasi Rekurensi

Deskripsi Tugas

Relasi rekurensi (*recurrence relation*) adalah sebuah tipe relasi matematis dimana definisi dari sebuah fungsi atau barisan dinyatakan secara rekursif, artinya merujuk pada fungsi atau barisan itu sendiri. Pada bagian ini, kalian akan berlatih untuk memahami definisi relasi rekurensi dan bagaimana menerapkannya, serta membuat definisi rekursif dari sebuah permasalahan.

1. Tentukan suku ke-10 dari barisan yang didefinisikan sebagai berikut:

$$a_i = \begin{cases} 1 & \text{jika } i = 1 \text{ atau } i = 2 \\ 2a_{i-1} + 2a_{i-2} & \text{jika } i > 2 \end{cases}$$

atau dengan kata lain, barisan tersebut dimulai dengan nilai 1, 1, kemudian untuk menghitung suku berikutnya, kita jumlahkan antara suku sebelumnya dengan dua kali dari suku sebelum suku sebelumnya.

2. Faktorial dari sebuah bilangan bulat $n \geq 1$, ditulis sebagai $n!$, didefinisikan sebagai sebuah nilai yang dihitung dengan mengalikan semua bulat dari 1 sampai dengan n .

Sebagai contoh, faktorial dari 5 adalah $5! = 1 \times 2 \times 3 \times 4 \times 5 = 120$. Buatlah sebuah definisi rekursif untuk menghitung nilai $n!$.



Ayo Berlatih!

Aktivitas Individu

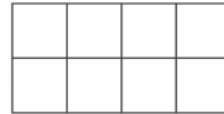
Aktivitas SAP-K11-03-U: Menerapkan Konsep Rekursi

Deskripsi Tugas

Selesaikanlah dua problem berikut dengan menerapkan konsep rekursi yang telah kalian pelajari. Setelah mengerjakan problem tersebut, diskusikanlah solusi kalian dengan teman.

Permasalahan 1: Memasang Keramik

Terdapat sebuah lantai yang berukuran $2 \times N$. Pada lantai tersebut, ingin dipasang N buah keramik, yang masing-masing berukuran 1×2 (perhatikan ilustrasi pada [gambar 2.6](#)).



Lantai



Keramik

▲ Gambar 2.6 Ilustrasi memasang keramik

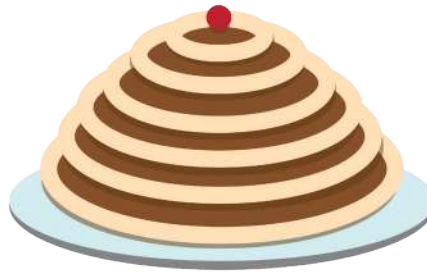
Setiap keramik dapat dipasang secara mendatar (horizontal) ataupun secara tegak (vertikal). Sebagai contoh, jika $N=4$, maka akan ada 5 buah cara berbeda memasang 4 keramik, yaitu:

- Semua keramik dipasang secara horizontal.
- Semua keramik dipasang secara vertikal.
- Paling kiri ada 2 keramik vertikal, sisanya horizontal.
- Paling kiri ada 2 keramik horizontal, sisanya vertikal.

- Paling kiri dan paling kanan vertikal, sisanya horizontal.
- Tentukan ada berapa cara memasang keramik untuk $N=8$?

Permasalahan 2: Menumpuk Panekuk

Budi memiliki setumpuk panekuk yang memiliki ukuran dari besar sampai kecil. Panekuk-panekuk tersebut ditumpuk di atas sebuah piring, dengan aturan bahwa panekuk yang besar harus berada di bawah panekuk yang lebih kecil.



▲ Gambar 2.7 Kue Panekuk

Budi ingin memindahkan panekuk ini dari satu piring ke piring lainnya, namun dalam prosesnya ia tetap ingin mengikuti aturan bahwa panekuk yang besar harus selalu berada di bawah panekuk yang lebih kecil. Selain itu, Budi juga hanya boleh memindahkan satu buah panekuk saja, pada satu waktu tertentu, dari satu piring ke piring lainnya. Budi menyadari bahwa ia memerlukan sebuah piring tambahan untuk dapat melakukan perpindahan ini. Jika piring asal panekuk diberi nama A, piring tujuan diberi nama C, maka Budi akan menyiapkan sebuah piring bantuan sebagai tempat sementara, yang diberi nama piring B.

Budi ingin mengetahui, berapa banyak langkah pemindahan panekuk yang harus ia lakukan, untuk dapat memindahkan semua panekuk yang dimilikinya, dari piring A ke piring C (dengan mungkin menggunakan piring B sebagai tempat sementara). Misalnya, jika Budi memiliki 2 buah panekuk saja (diberi nama panekuk 1 dan, dimana panekuk 1 berukuran lebih kecil dari panekuk 2), maka ia akan membutuhkan minimal 3 langkah pemindahan:

1. Pindahkan panekuk 1 dari piring A ke piring B
2. Pindahkan panekuk 2 dari piring A ke piring C
3. Pindahkan panekuk 1 dari piring B ke piring C

Berapakah jumlah langkah minimal yang diperlukan apabila Budi memiliki 6 buah panekuk?



Ayo Renungkan!

Setelah selesai melakukan aktivitas tersebut. Jawablah pertanyaan berikut ini dalam Lembar Refleksi pada Buku Kerja, dan jangan lupa mencatat kegiatan dalam Jurnal Peserta Didik.

1. Apakah kalian dapat menentukan nilai dari suatu barisan/fungsi, apabila diberikan nilai awal dan definisi relasi rekurensi-nya?
2. Apakah kalian sudah berhasil memodelkan contoh masalah pada latihan-latihan di atas dalam bentuk relasi rekurensi?
3. Apakah kalian merasa ada masalah lain yang serupa (dalam kehidupan sehari-hari) yang dapat dimodelkan dalam bentuk relasi rekurensi tadi?
4. Pelajaran paling berkesan apa yang kalian dapatkan dari aktivitas latihan ini?

2. Algoritma Greedy

Greedy secara harfiah berarti rakus atau tamak. Meskipun dalam pengertian sehari-hari, kata “rakus” dan “tamak” memiliki konotasi negatif, namun dalam konteks Informatika, kita mengartikan *greedy* dalam konteks sebagai sebuah strategi penyelesaian masalah yang dapat berguna dalam merancang sebuah algoritma atau solusi bagi sebuah permasalahan komputasional. Oleh karena itu, diharapkan tidak ada konotasi negatif pada kata *greedy* dalam konteks ini.

Teknik *greedy* adalah salah satu teknik penyelesaian masalah yang biasa digunakan untuk menyelesaikan permasalahan optimasi. Permasalahan optimasi berarti kita ingin menghitung sebuah hasil yang terbaik dari sebuah proses tertentu. Terbaik disini dapat berarti nilai yang paling kecil ataupun paling besar, tergantung dari jenis permasalahannya. Dalam menyelesaikan permasalahan optimasi seperti ini, algoritma *greedy* akan menerapkan prinsip “mengambil serangkaian langkah terbaik pada setiap saat”.

Contoh 1: Membawa Ikan 1

Budi ingin membawa beberapa ekor ikan yang sudah tersimpan dalam kantong-kantong plastik untuk diangkut di dalam mobilnya. Terdapat 8 buah kantong dengan yang berisi masing-masing 3, 5, 2, 8, 4, 6, 6, dan 3 ekor ikan. Namun sayangnya, mobilnya hanya mampu membawah 4 buah kantong. Kantong-kantong manakah yang harus dibawa oleh Budi agar jumlah ikan yang dibawanya sebanyak mungkin?

Jawab:

Untuk dapat membawa sebanyak mungkin ikan, Budi harus memilih kantong-kantong dengan sebanyak mungkin ikan. Oleh karena itu, algoritma *greedy* dapat diterapkan disini, dengan cara kita mengambil kantong mulai dari yang berisi ikan paling banyak terlebih dahulu, sampai didapatkan 4 buah kantong. Dengan demikian, kita harus mengurutkan kantong-kantong terlebih dahulu mulai dari yang paling banyak ikannya, sampai dengan yang paling sedikit, sehingga urutannya menjadi: 8, 6, 6, 5, 4, 3, 3, 2. Jika kita ambil 4 buah kantong pertama, maka total banyaknya ikan yang dapat dibawa adalah $8 + 6 + 6 + 5 = 25$ ekor ikan. Tentunya tidak ada pilihan 4 kantong yang akan menghasilkan total banyaknya ikan lebih dari 25 ekor.

Contoh 2: Membawa Ikan 2

Kali ini, Budi harus membawa sedikitnya 15 ekor ikan. Tentukan jumlah kantong terkecil yang harus dibawa oleh Budi, agar terdapat minimal 15 ekor ikan yang terbawa!

Jawab:

Sama seperti pada permasalahan sebelumnya, kita dapat menerapkan algoritma greedy untuk menyelesaikan permasalahan ini. Dalam hal ini, untuk memperkecil banyaknya kantong yang harus dibawa, maka kita juga selalu memilih kantong dengan jumlah ikan terbanyak terlebih dahulu. Jika kita memilih kantong dengan jumlah ikan = 8 dan 6, maka kita sudah memiliki 14 ekor ikan. Selanjutnya, kita hanya perlu mengambil 1 kantong lagi (yang mana saja) agar total jumlah ikan menjadi lebih dari 15. Oleh karena itu, jawaban yang diinginkan adalah 3 buah kantong. Jelas bahwa tidak ada pilihan yang memungkinkan kita mendapatkan 15 ekor ikan dengan 2 atau kurang kantong.

Pada kedua contoh di atas, terdapat satu langkah yang penting yang biasa diterapkan pada penyelesaian masalah secara *greedy*, yaitu proses mengurutkan sebuah data agar menjadi terurut (mungkin dari kecil ke besar, atau sebaliknya), agar kemudian kita dapat melakukan serangkaian pengambilan langkah secara *greedy* pada data yang sudah terurut tersebut. Pola seperti ini umum digunakan pada penyelesaian permasalahan secara *greedy*.



Ayo Berlatih!

Aktivitas Kelompok

Aktivitas SAP-K11-04-U: Mengerjakan Pekerjaan Rumah (PR)

Deskripsi Tugas

Cici menerima 10 buah pekerjaan rumah (PR) yang harus ia kerjakan. Setelah melihat isi dari masing-masing PR, Cici memiliki perkiraan, berapa lama waktu yang diperlukan untuk mengerjakan masing-masing PR tersebut, seperti terlihat pada tabel di bawah.

No.	PR	Waktu pengerjaan (jam)
1.	A	1.5
2.	B	3
3.	C	1
4.	D	0,5
5.	E	4

No.	PR	Waktu pengerjaan (jam)
6.	F	1
7.	G	2,5
8.	H	1
9.	I	0,5
10.	J	2

Sayangnya, ia tidak punya banyak waktu untuk mengerjakan semua PR. Cici menghitung bahwa ia hanya punya waktu total = 8 jam sebelum semua PR tersebut harus dikumpulkan. Cici ingin menentukan, PR mana yang harus ia kerjakan terlebih dahulu, dengan pertimbangan bahwa setiap PR memiliki nilai yang sama besarnya (terhadap nilai akhir Cici). Bantulah Cici menentukan PR yang mana saja yang harus ia kerjakan dalam waktu maksimal 8 jam, untuk mendapatkan total nilai akhir yang sebesar-besarnya.



Ayo Berlatih!

Aktivitas Individu

Aktivitas SAP-K11-05-U: Mengunjungi Kebun Binatang

Deskripsi Tugas

Dina sedang bertamasya mengunjungi kebun binatang. Setiap hari, kebun binatang mengadakan beberapa pertunjukan atraksi hewan yang dapat ditonton oleh para pengunjung. Berikut adalah jadwal yang telah ditetapkan oleh pengelola kebun binatang.

No.	Waktu Mulai	Waktu Selesai	Pertunjukan/Atraksi Hewan
1.	9:15	10:30	Orang Utan
2.	8:00	9:30	Pinguin
3.	10:00	12:00	Harimau
4.	13:00	14:30	Beruang Madu
5.	11:00	12:30	Burung Pemangsa
6.	14:00	15:00	Buaya
7.	15:30	16:30	Panda
8.	16:00	17:00	Ular Piton
9.	15:00	15:30	Singa
10.	15:30	16:00	Anjing Laut

Tentunya dalam satu waktu tertentu, Dina hanya dapat menonton satu pertunjukan atraksi hewan. Dina ingin dapat melihat sebanyak-banyaknya pertunjukan dalam satu hari tersebut, dan ia tidak memiliki preferensi dalam melihat pertunjukan hewan (semuanya ia anggap sama menariknya). Tentukan, ada berapa banyak maksimal pertunjukan yang dapat ditonton oleh Dina?



Ayo Berlatih!

Aktivitas Individu

Aktivitas SAP-K11-06-U: Menukarkan Uang

Deskripsi Tugas

Dalam kehidupan sehari-hari, kita pasti sudah banyak terbiasa dengan perhitungan yang melibatkan uang. Misalnya, ketika Anda membeli sebuah barang/makanan, atau pun ingin membayar untuk sebuah jasa tertentu, kita seringkali menyiapkan sejumlah uang tertentu, sesuai dengan harga barang atau jasa tersebut. Selanjutnya, bagi penjual atau penyedia jasa, apabila mereka menerima uang pembayaran dengan jumlah total yang lebih besar dari harga yang ditetapkan, mereka pun juga harus menyiapkan uang kembalian sesuai dengan jumlah kelebihan pembayaran.

Di Indonesia, mata uang rupiah memiliki beberapa pecahan uang, mulai dari yang terkecil, 100 Rupiah, 200 Rupiah, 500 Rupiah, dst sampai dengan 100 000 Rupiah. Seandainya kita memiliki sejumlah pecahan uang, misalnya beberapa uang seribuan, dua ribuan, lima ribuan, sepuluh ribuan dan dua puluh ribuan. Jika kita ingin mendapatkan uang tepat sejumlah 38 ribu rupiah, maka kita dapat memilih beberapa cara, misalnya:

- 3 lembar sepuluh ribuan, ditambah 1 lembar lima ribuan, ditambah 2 lembar ribuan, ditambah 2 koin lima ratus, dengan total ada 8 buah lembaran uang/koin
- 1 lembar dua puluh ribuan, ditambah 1 lembar sepuluh ribuan ditambah 4 lembar dua ribuan, totalnya menjadi 6 lembaran uang
- 1 lembar dua puluh ribuan, ditambah 1 lembar sepuluh ribuan, ditambah 1 lembar lima ribuan ditambah 1 lembar dua ribuan, ditambah 1 lembar seribuan, dengan total ada 5 lembaran uang.

Jelas bahwa jumlah total lembaran yang dibutuhkan tergantung dari pemilihan pecahan uang yang kita gunakan. Nah, permasalahan yang mungkin kita tanyakan adalah: Bagaimana caranya memilih pecahan-pecahan uang yang akan digunakan sedemikian rupa, sehingga total lembaran yang diperlukan untuk menghasilkan suatu nilai uang tertentu menjadi sekecil mungkin?

Pada contoh di atas, dapat diperiksa bahwa untuk menghasilkan nilai uang sebesar tiga puluh delapan ribu rupiah dari pecahan-pecahan seribuan, dua ribuan, lima ribuan, sepuluh ribuan dan dua puluh ribuan, maka diperlukan minimal 5 buah lembar, yaitu sesuai dengan cara terakhir di atas. Dapatkah Anda mencari strategi yang umum untuk menyelesaikan permasalahan serupa, jika jumlah nilai uang yang dihasilkan berbeda (namun dengan pecahan-pecahan uang yang sama)? Kita bisa menganggap bahwa jumlah nilai yang diinginkan selalu merupakan kelipatan ribuan rupiah (sehingga selalu bisa didapatkan dengan menggabungkan pecahan-pecahan di atas).



Ayo Renungkan!

Setelah selesai melakukan aktivitas di atas, jawablah pertanyaan berikut ini dalam Lembar Refleksi pada Buku Kerja, dan jangan lupa mencatat kegiatan dalam Jurnal Peserta Didik.

1. Apakah kalian dapat memberikan sebuah contoh lain dari permasalahan optimasi yang ada di kehidupan sehari-hari?
2. Untuk contoh permasalahan yang kalian pilih sebagai jawaban di no.1, menurut kalian apakah algoritma *greedy* dapat diterapkan pada permasalahan tersebut?
3. Pada permasalahan penukaran uang di atas, apakah algoritma *greedy* selalu dapat digunakan untuk mencari jawaban yang paling optimal? Ambil sebuah contoh

kasus dimana pecahan yang tersedia hanya 1000, 7000 dan 10000, dan kita ingin menukarkan nilai 15.000. Berapakah jawaban yang diberikan oleh algoritma *greedy* pada kasus seperti ini? Apakah ini adalah jawaban yang optimal?

4. Untuk kasus pada pertanyaan no 3, menurut kalian, strategi seperti apakah yang kira-kira lebih tepat untuk digunakan?
5. Pelajaran paling berkesan apa yang kalian dapatkan dari latihan ini?

3. Pemrograman Dinamis

Saat menyelesaikan sebuah permasalahan optimasi (mencari nilai terbesar/terkecil), terkadang kita harus memperhitungkan beberapa kemungkinan pengambilan langkah untuk menyelesaikan permasalahan tersebut. Kemungkinan-kemungkinan tersebut mungkin memiliki akibat/konsekuensi terhadap langkah-langkah selanjutnya, sehingga pendekatan seperti teknik *greedy* mungkin tidak akan menghasilkan jawaban yang optimal. Dalam hal ini, teknik pemrograman dinamis atau *dynamic programming* (DP) mungkin akan lebih sesuai diterapkan. Teknik DP mengandung dua unsur utama, yaitu:

1. Optimasi (mencari nilai terkecil/terbesar) melalui serangkaian pilihan. Serupa dengan teknik *greedy*, kita harus menentukan rangkaian langkah apa yang akan menghasilkan nilai optimal di akhir. Namun, berbeda dengan permasalahan yang dapat diselesaikan dengan teknik *greedy*, permasalahan yang sesuai untuk teknik DP memiliki struktur sedemikian rupa sehingga pilihan langkah terbaik saat ini belum tentu merupakan pilihan terbaik secara keseluruhan, sehingga prinsip *greedy* belum tentu dapat diterapkan, dan semua kemungkinan kombinasi pilihan langkah harus diperhitungkan.

2. Nilai optimal yang diinginkan untuk permasalahan tersebut biasanya dapat dinyatakan sebagai kombinasi optimal dari sub-sub permasalahan yang sama, tetapi dengan ukuran yang lebih kecil (atau dengan kata lain, dapat dinyatakan secara rekursif). Namun, sub-sub permasalahan yang harus dipertimbangkan, biasanya memiliki *overlap* (persinggungan) sehingga dalam proses perhitungannya, diperlukan cara yang efisien untuk menghitung solusi untuk sub-sub permasalahan yang diperlukan, agar tidak terjadi perulangan/duplikasi dalam proses perhitungan. Cara yang umum digunakan adalah dengan menyimpan semua solusi dari *subproblem* yang sudah diketahui dalam sebuah tempat penyimpanan/ tabel. Teknik ini biasa disebut sebagai teknik memorisasi.

Contoh 1:

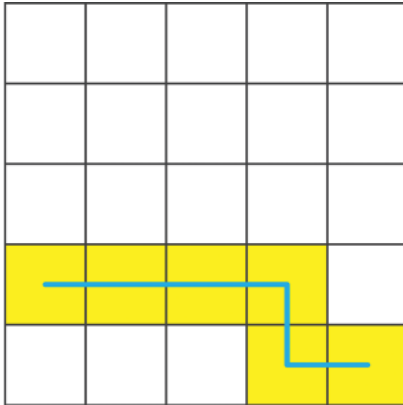
Agria ingin memanen tanaman cabai di halaman rumahnya. Tanaman tersebut ditata dalam bentuk kotak-kotak persegi seperti ilustrasi di bawah ini. Angka pada setiap kotak mewakili jumlah cabai yang ada di masing-masing tanaman.

0	1	2	3	10
10	2	10	10	5
5	3	6	5	0
0	4	0	2	3
20	5	1	0	0

▲ Gambar 2.8 Ilustrasi Jumlah Cabai yang dipanen Agria

Agria tidak punya waktu banyak karena ia harus segera pergi ke kampus. Oleh karena itu, ia tidak bisa memetik seluruh cabai tersebut. Ia hanya bisa mulai dari kotak manapun di kolom paling kiri, dan berhenti di kotak manapun di kolom paling kanan. Agria hanya bisa bergerak ke kotak di tepat setelah kanannya

atau bawahnya. Berikut adalah salah satu dari sekian banyak kemungkinan jalur yang dapat dilalui oleh Agria untuk memetik cabai.



▲ Gambar 2.9 Kemungkinan Jalur Memetik Cabai Agria

Berapakah jumlah cabai terbanyak yang bisa dikumpulkan oleh Agria?

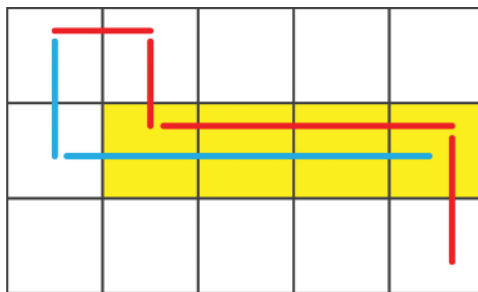
Jawab:

Pertama, perlu dipahami bahwa penggunaan teknik *greedy* pada permasalahan ini tidak akan menghasilkan jawaban yang benar/optimal. Dapat dilihat bahwa jika kita menggunakan prinsip *greedy*, maka kita akan memilih untuk memulai dari kolom pertama baris terakhir, dengan nilai jumlah cabai terbesar, yaitu 20. Namun, jika kita memulai dari sini, maka tidak ada pilihan lain untuk langkah-langkah selanjutnya, selain bergerak terus ke kanan. Maka nilai total cabai yang akan didapatkan adalah $= 20 + 5 + 1 + 0 + 0 = 26$. Jelas bahwa ada pilihan-pilihan jalur lain yang akan menghasilkan total nilai cabai > 26 , misalnya langkah sebagai berikut akan menghasilkan jumlah total cabai $= 0 + 10 + 2 + 10 + 10 + 5 = 37$.

0	1	2	3	10
10	2	10	10	5
5	3	6	5	0
0	4	0	2	3
20	5	1	0	0

▲ Gambar 2.10 Kemungkinan Jalur Lain Memetik Cabai Agria

Metode berikutnya yang bisa kita pikirkan solusinya adalah mencoba semua kemungkinan jalur, lalu menghitung berapa nilai total cabai yang bisa didapatkan, dan terakhir mencari nilai terbesarnya. Namun dengan metode ini kita menemui kendala lainnya, yaitu akan ada terlalu banyak kemungkinan yang harus kita perhitungkan. Satu hal yang dapat kita segera pahami adalah bahwa ada banyak sekali persinggungan antara jalur-jalur yang berbeda, sedemikian rupa sehingga akan ada banyak sekali perulangan yang tidak perlu, ketika kita menghitung nilai total 0 cabai dari semua kemungkinan jalur yang ada. Sebagai contoh, kedua jalur di bawah ini (*jalur biru dan jalur merah*), akan melalui empat kotak yang sama dan menghitung penjumlahan dari nilai di keempat kotak yang sama tersebut (*warna kuning*).



▲ Gambar 2.11 Contoh Ilustrasi Persinggungan antara dua Kemungkinan Jalur (Overlap)

Menggunakan prinsip DP, yang perlu kita lakukan adalah pertama-tama menyatakan solusi/penyelesaian dari permasalahan awal sebagai kombinasi dari sub-permasalahan yang lebih kecil. Dalam hal ini, kita dapat membuat argumentasi bahwa nilai jumlah cabai terbanyak yang bisa kita kumpulkan sampai dengan suatu kotak tertentu (di manapun kolomnya) tergantung dari nilai terbaik jumlah cabai sampai dengan kotak di atasnya, atau kotak di sebelah kirinya (jika ada), dan tinggal kita jumlahkan saja dengan nilai banyaknya cabai di kotak akhir tersebut. Hal ini, tentunya karena kita hanya bisa bergerak ke kanan atau ke bawah saja. Misalnya, pada [Gambar 2.12](#) ini:

				A
			B	C

▲ **Gambar 2.12** Ilustrasi Prinsip Pemrograman Dinamis pada Persoalan Panen Cabai Agria

Nilai terbaik yang bisa kita dapatkan akan berakhir pada kotak berwarna hitam, dapat dihitung dengan cara menghitung nilai terbaik yang didapatkan sampai dengan kotak merah (misalkan nilainya = A), dan sampai dengan kotak warna biru (misalkan nilainya = B). Maka, untuk mendapatkan nilai terbaik sampai dengan kotak warna hitam, kita hanya mencari, manakah jumlah yang tertinggi antara nilai A dan B, kemudian nilai tersebut dijumlahkan dengan nilai C.

Proses di atas mengubah permasalahan ini menjadi bersifat rekursif, dimana kita bisa menggunakan hasil perhitungan pada kotak-kotak sebelumnya untuk menghitung nilai terbaik pada kotak-kotak selanjutnya (yang berada di posisi

lebih ke kanan atau ke bawah), sehingga dengan cara ini, kita bisa menghindari perulangan (duplikasi) proses perhitungan. Proses ini biasanya menggunakan sebuah tabel perhitungan yang biasa disebut sebagai tabel memoisasi (atau tabel DP). Istilah **memoisasi** berasal dari bahasa latin *memorandum* yang berarti “mengingat”, yang kemudian biasa disingkat sebagai memo dalam Bahasa Inggris. Harap bedakan istilah memoisasi ini dengan memorisasi (*memorization*) yang juga memiliki arti yang serupa (proses mengingat), namun memoisasi memiliki arti yang khusus dalam dunia komputasi, yaitu: menyimpan/mengingat hasil perhitungan yang telah dilakukan sebelumnya, sehingga tidak perlu mengulang perhitungan yang sama dua kali.

Untuk soal ini, kita buat tabel memoisasi tersebut sebagai berikut:

1. Kotak paling kiri atas kita berikan nilai = nilai isi kotak tersebut (0)
2. Untuk setiap kotak lainnya, misalkan A = nilai yang sudah dihitung pada tabel memoisasi untuk kotak yang ada di atasnya (atau 0 jika kotak saat ini ada di baris teratas), dan B = nilai yang sudah dihitung pada tabel memoisasi untuk kotak yang ada di sebelah kirinya (atau 0 jika kotak saat ini ada di kolom paling kiri), serta misalkan C = nilai cabai yang ada pada kotak saat ini. Maka kita isi kotak saat ini pada tabel memoisasi dengan nilai $\max(A, B) + C$.
3. Kita lakukan proses di atas sampai tabel memoisasi terisi penuh (sesuai ukuran tabel nilai cabai di awal). Nilai paling besar pada tabel memoisasi menunjukkan nilai total jumlah cabai terbesar yang bisa dikumpulkan.

Hasil tabel memoisasi yang sudah terisi penuh untuk soal di atas adalah sebagai berikut:

▼ Tabel 2.2 Memoisasi Persoalan Panen Cabai Agria

0	1	3	6	16
10	12	22	32	37
15	18	28	37	37
15	22	28	39	42
35	40	41	41	42



Ayo Berlatih!

Aktivitas Berpasangan

Aktivitas SAP-K11-07-U: Bermain Angka

Deskripsi Tugas

Ani dan Budi sedang bermain dengan sebuah permainan angka: pertama Ani akan memilih sebuah angka bilangan bulat positif n . Selanjutnya, Budi harus mengubah bilangan n ini menjadi angka 1 dengan menerapkan serangkaian langkah sebagai berikut:

1. Budi boleh mengganti bilangan n dengan $n - 1$.
2. Jika bilangan saat ini adalah genap (habis dibagi 2), maka Budi boleh menggantinya dengan $n/2$.
3. Jika bilangan saat ini habis dibagi 3, maka Budi boleh menggantinya dengan $n/3$.

Proses ini harus dilakukan oleh Budi secara terus menerus sampai bilangan yang dimilikinya menjadi 1. Misalnya, jika Ani memilih $n = 5$, maka Budi dapat melakukan proses mengubah 5 menjadi 1 sebagai berikut: $5 \rightarrow 4 \rightarrow 2 \rightarrow 1$ (dalam tiga langkah). Tentukan, berapakah jumlah langkah minimum yang diperlukan, jika Ani memilih $n = 25$?



Ayo Renungkan!

Setelah selesai melakukan aktivitas tersebut. Jawablah pertanyaan berikut ini dalam Lembar Refleksi pada Buku Kerja, dan jangan lupa mencatat kegiatan dalam Jurnal Peserta Didik.

1. Menurut kalian, apakah permasalahan penukaran uang pada aktivitas SAP-K11-06-U: Menukarkan Uang untuk topik greedy sebelumnya (yaitu pecahan yang tersedia hanya 1000, 7000 dan 10000, dan kita ingin menukarkan nilai 15.000) dapat diselesaikan dengan teknik pemrograman dinamis? Bagaimana caranya?
2. Apakah ada contoh permasalahan yang lain dalam kehidupan sehari-hari yang dapat diselesaikan dengan teknik pemrograman dinamis (atau setidaknya, dimana teknik memoisasi dapat diterapkan)?
3. Menurut Anda, apakah yang menjadi perbedaan penting antara algoritma greedy dan pemrograman dinamis? Adakah kekurangan dan kelebihan masing-masing?
4. Manakah pelajaran yang paling berkesan dari topik ini?

C. Algoritma dan Pemrograman

Pada jenjang sebelumnya dalam materi Informatika Kelas X, kalian telah mempelajari mengenai beberapa bahasa pemrograman, salah satunya adalah **bahasa C** yang telah dipelajari pada elemen Algoritma dan Pemrograman. Kemampuan bahasa pemrograman terutama bahasa C akan kalian gunakan dalam topik ini. Selanjutnya untuk lebih meningkatkan pemahaman kalian dalam elemen Algoritma dan Pemrograman, kalian akan mempelajari konsep larik serta karakter dan string. Kedua konsep ini sangat penting dalam membuat program dan akan kalian gunakan untuk menyelesaikan berbagai permasalahan yang diberikan pada bagian D dalam bab ini.

1. Larik (Array)

Saat ini mungkin kalian memiliki pertanyaan seperti “mengapa contoh dan permasalahan yang diberikan selama ini adalah hal yang dapat diselesaikan oleh manusia secara manual atau dengan kalkulator?” Jawabannya adalah karena permasalahan/*problem* tersebut diberikan untuk mendukung proses kalian menguasai kompetensi dalam algoritma dan pemrograman.

Pada praktiknya, program digunakan untuk mengolah data yang berukuran besar dan membutuhkan waktu yang sangat lama jika dikerjakan manual oleh manusia. Misalnya kalian perlu menghitung statistika deskriptif (seperti rata-rata, nilai minimal, nilai maksimal, standar deviasi, dan sebagainya) dari data seluruh penduduk Indonesia. Walaupun masalahnya sederhana, namun dikarenakan jumlah data yang diolah sangat banyak dan berukuran besar mengakibatkan waktu pengerjaan pun menjadi sangat lama bagi manusia untuk menyelesaikannya. Bahkan ada kemungkinan data berukuran besar tersebut tidak dapat diolah menggunakan aplikasi pengolah data (*spreadsheet*) yang tidak dirancang untuk mengolah data sebesar itu. Untuk solusinya kalian dapat menggunakan program khusus untuk mengolah data berukuran besar atau membuat program sendiri yang mampu menyimpan dan mengolah data berukuran besar.

Kita pun sampai pada pertanyaan besar: “bagaimana caranya membuat program yang mampu menyimpan dan mengolah data berukuran besar?”. Sebelumnya kalian telah mengenal konsep variabel yang mampu menyimpan satu buah nilai dengan tipe data tertentu (variabel tunggal). Permasalahan akan muncul ketika program kita harus mengolah sebanyak satu juta data, apakah kita harus membuat satu juta variabel? Bukankah hal tersebut sangat sulit dipraktikkan dalam kode program yang kita tulis? Untuk mengatasi hal tersebut, bahasa pemrograman

memiliki suatu alat untuk menyimpan himpunan data ke dalam satu nama variabel yang diberikan indeks. Salah satunya disebut sebagai **larik** atau *array*.

Contoh di dunia nyata yang merepresentasikan larik adalah seperti loker yang diberikan nomor (**Gambar 2.13**). Ketika kalian menyimpan barang di loker tersebut, kalian akan mengingat nomor loker tersebut. Kalian juga dapat menyimpan barang di loker dengan nomor yang berbeda. Pada analogi tersebut, nomor pada loker adalah indeks yang kita gunakan untuk mengenali tempat kita menyimpan barang kita tadi.



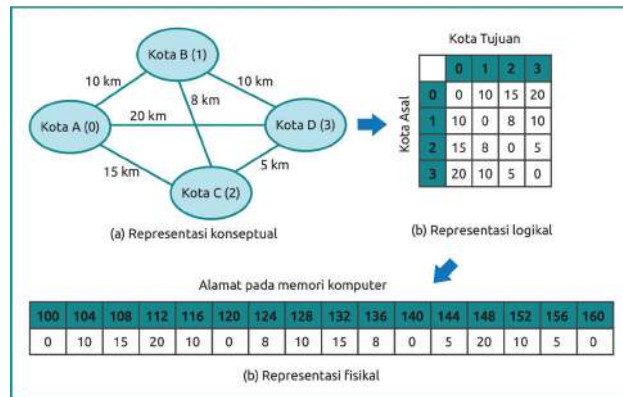
▲ **Gambar 2.13** Loker sebagai Ilustrasi Larik dalam Pemrograman

Menggunakan larik pada program mirip dengan membuat variabel tunggal. Perbedaannya adalah kalian perlu memberikan informasi ukuran dari larik yang akan dibuat dan kalian perlu mengaksesnya dengan menggunakan indeks. Ada banyak cara untuk mengimplementasikan konsep larik ke dalam kode program, salah satunya menggunakan *vector*. *Vector* ini merupakan pustaka bahasa pemrograman C++ yang juga dapat digunakan pada bahasa C. *Vector* dapat dibuat untuk menampung himpunan data yang ukurannya dinamis bergantung pada jumlah (atau ukuran) data yang disimpan di dalamnya.

Contoh kode program di bawah ini mengilustrasikan cara mendeklarasikan, menginisialisasi, menyimpan nilai ke, dan mengakses nilai pada suatu *vector*. Berbeda dengan buku Kelas X, program ini ditulis menggunakan bahasa pemrograman C++. Oleh karena itu, kalian perlu menyimpan kode program dengan ekstensi berkas .cpp dan mengkompilasinya dengan kompilator bahasa C++, misalnya [GCC](#).

```
1:  /* Demonstrasi larik satu dimensi dengan vektor */
2:
3:  #include <stdio.h>
4:  #include <vector>
5:  using namespace std;
6:
7:  void cetakLarik(int ukuran, vector<int> larik){
8:      int i;
9:      for(i = 0; i < ukuran; i++)
10:         printf("%d\n", larik[i]);
11:  }
12:
13:  int main(){
14:      vector<int> bilangan;
15:      int i;
16:      for(i = 0; i < 10; i++)
17:         bilangan.push_back(i);
18:      cetakLarik(10, bilangan);
19:  }
```

Kalian juga dapat membuat larik yang berukuran dua dimensi, yang memiliki ukuran berupa baris dan kolom (seperti pada matriks di matematika). Pada kasus seperti apa larik dua dimensi digunakan? Misalnya kalian ingin menghitung jarak antara dua kota yang dihubungkan dengan jalan dua arah. Kalian dapat menggunakan matriks dua dimensi untuk merepresentasikan jarak antara dua kota seperti pada [Gambar 2.14](#) berikut.



Gambar 2.14 Ilustrasi representasi panjang jalan penghubung antar empat kota (A-D) dalam bentuk matriks dua dimensi.

Gambar tersebut memperlihatkan proses berpikir komputasional yaitu abstraksi. Suatu hubungan antara kota di dunia nyata (kontekstual) dibuat dalam bentuk yang lebih sederhana dalam bentuk grafik yang menampilkan informasi yang relevan (konseptual), yaitu hubungan antar kota dan jaraknya. Setelah itu, bentuk tadi diubah ke dalam bentuk yang dapat digunakan dalam menyusun algoritma komputer (logikal) dalam bentuk matriks dua dimensi. Di balik program yang berjalan di komputer kalian, program akan menyimpan matriks tersebut di dalam memori komputer (fisikal) yang memiliki bentuk seperti matriks satu dimensi.

Implementasi larik bisa berbeda-beda di bahasa C atau bahasa lain, yang penting adalah implementasi dari kode program tersebut memenuhi perilaku larik, yaitu adanya himpunan data yang disimpan dan dapat diakses dengan menggunakan suatu indeks. Misalnya, kalian kemudian dapat membuat kode program yang akan menerima isi matriks jarak di atas dan menyimpannya dalam sebuah vector dua dimensi. Setelah itu, program akan dapat menjawab jarak antara dua kota yang ditanyakan oleh pengguna. Perhatikanlah potongan kode program berikut. Setelah itu, perkuat pemahaman kalian dengan latihan yang diberikan.

```

1:  /* Program demonstrasi penggunaan larik dua dimensi */
2:
3:  #include <stdio.h>
4:  #include <vector>
5:  #define JUMLAH_KOTA 4
6:  using namespace std;
7:
8:  int main(){
9:      vector<vector<int>> jarak;
10:     int i, j;
11:     int asal, tujuan;
12:
13:     for(i = 0; i < JUMLAH_KOTA; i++){
14:         vector<int> vtr;
15:         for(j = 0; j < JUMLAH_KOTA; j++){
16:             int x;
17:             scanf("%d",&x);
18:             vtr.push_back(x);
19:         }
20:         jarak.push_back(vtr);
21:         vtr.clear();
22:     }
23:
24:     scanf("%d %d", &asal, &tujuan);
25:     printf("Jarak Kota-%d Kota -%d = ", asal, tujuan);
26:     printf("%d km\n", jarak[asal][tujuan]);
27: }

```



Ayo Berlatih!

Aktivitas Individu

Aktivitas SAP-K11-08: Latihan Larik

1. Rancang dan buatlah sebuah program yang membaca N buah bilangan dan mencetaknya secara terbalik. Misalnya jika diberikan masukan 1 2 3 4 5 6, program akan mencetak 6 5 4 3 2 1.
2. Rancang dan buatlah sebuah program yang membaca sebuah matriks berukuran $N \times M$ dan mencetak hasil *transpose* matriksnya.
3. Modifikasilah kode program untuk menghitung jarak dua kota yang menerima suatu rute. Misalnya, jika rute yang dimasukkan adalah Kota A - Kota B - Kota C, maka nilai total jarak yang dikeluarkan adalah $10 + 8 = 18$ km.

4. Tantangan larik: perhatikan bahwa pada Gambar 2.9 terdapat data yang berganda. Hal ini dikarenakan matriks tersebut menyimpan informasi jarak antara 2 kota (misal A dan B) sebagai jarak dari A-B dan B-A. Dapatkan kalian menemukan representasi yang lebih baik daripada contoh tersebut sehingga tidak ada duplikasi informasi dalam penyimpanan data jarak kota?

Ingin
Tahu Lebih?



1. Masih banyak detail dan fitur dari vector yang tidak dapat dijelaskan lengkap di buku ini. Kalian dapat membacanya lebih lanjut di dokumentasi vector di bahasa C++ berikut: en.cppreference.com/w/cpp/container/vector
2. Selain menggunakan vector, larik pada bahasa C dapat diimplementasikan menggunakan array bawaan dari bahasa pemrograman C. Dokumentasi larik dalam bahasa C dapat diakses di en.cppreference.com/w/c/language/array.
3. Larik adalah cara yang paling sederhana untuk menyimpan himpunan data. Masih ada banyak cara lain untuk menyimpan dan mengolah data, yang kita sebut sebagai struktur data. Kalian dapat melihat ilustrasi visual dari bagaimana beberapa struktur data bekerja di: visualgo.net/id.

► Praktik Baik Pemrograman

Saat memprogram, kalian akan menggunakan suatu bahasa pemrograman, pustaka, atau perangkat tertentu. Biasakanlah untuk selalu melihat dokumentasi yang tersedia dari teknologi yang kalian gunakan. Dokumentasi ini disediakan oleh pengembang teknologi untuk memberikan informasi tentang teknologi yang mereka sediakan. Pada bahasa pemrograman misalnya kalian dapat melihat penjelasan sintaks dan fitur-fitur terkait bahasa pemrograman yang kalian gunakan. Ada banyak fitur menarik yang dapat kalian eksplorasi dan manfaatkan di program yang kalian buat.

2. Karakter dan *String*

Selain angka, masukan dari program dapat berupa karakter atau rangkaian karakter seperti kata. Hal ini telah lazim kalian temui, misalnya ketika mengetikkan kata kunci untuk melakukan pencarian di mesin pencari atau ketika kalian memasukkan kata sandi saat *login*. Oleh karena itu, program dilengkapi dengan kemampuan untuk membaca, menyimpan, mengolah, dan mencetak rangkaian karakter tersebut. Rangkaian karakter tersebut dalam pemrograman disebut sebagai *string*.

Definisi *string* dalam pemrograman adalah rangkaian karakter. Karakter sendiri merupakan suatu data berupa huruf, angka, simbol, dan karakter lain yang mengikuti suatu standar tertentu seperti “**American Standard Code for Information Interchange**” (ASCII) atau **Unicode**. Karakter pada pemrograman pada umumnya diimplementasikan dalam program menggunakan tipe data *char*. Di sisi lain, ada beberapa cara yang lazim digunakan untuk mengimplementasikan *string* dalam pemrograman.

Pada bahasa C, *string* diimplementasikan sebagai larik karakter yang diakhiri oleh karakter ‘\0’. Dengan kata lain, kalian dapat membuat dan memproses suatu *string* seperti halnya kalian mengolah larik. Bahasa pemrograman lain seperti C++ atau Java memilih sebuah tipe data *string* sendiri yang menyembunyikan beberapa detail terkait pengelolaan data *string* yang dilakukan oleh program.

Karena proses pada *string* berbeda dengan proses pada bilangan, untuk itu bahasa pemrograman telah dilengkapi dengan fungsi-fungsi untuk mengolah karakter dan *string*. Misalnya untuk melakukan konversi dari huruf kapital ke non kapital, penggabungan *string*, pencarian *substring*, dan berbagai fungsi lainnya. Pada bahasa C, kalian dapat akses fungsi-fungsi tersebut pada pustaka <string.h>. Pada contoh

berikut, diberikan dua buah kode program C++ untuk mengolah karakter dan string.

Program pertama akan membaca sebuah *string*, mengubahnya ke dalam huruf non kapital, lalu mencetaknya:

```
1:  /* Demonstrasi karakter dan string */
2:
3:  #include <stdio.h>
4:  #include <iostream>
5:  #include <string.h>
6:  using namespace std;
7:
8:  int main(){
9:      string str;
10:     cin >> str;
11:     for(int i = 0; i < str.length(); i++)
12:         str[i] = tolower(str[i]);
13:
14:     cout << str << '\n';
15: }
```

Program kedua akan membaca banyak *string* hingga membaca string “STOP”. Setiap *string* yang dibaca akan diubah ke huruf non kapital dan dicetak.

```
1:  /* Demonstrasi karakter dan string */
2:
3:  #include <stdio.h>
4:  #include <iostream>
5:  #include <string.h>
6:  using namespace std;
7:
8:  string lowerCase(string str){
9:     for(int i = 0; i < str.length(); i++)
10:        str[i] = tolower(str[i]);
11:     return str;
12: }
13:
14: int main(){
15:     string str;
16:     cin >> str;
17:     while(str != "STOP"){
18:         cout << lowerCase(str) << '\n';
19:         cin >> str;
20:     }
21: }
```

Telusurilah kedua program tersebut untuk memahami seperti apa mereka bekerja. Kemudian, salin dan jalankan kedua program tersebut menggunakan kompilator bahasa C++. Jika diperlukan, pelajari dokumentasi pustaka `string.h` di C dan tipe data `string` pada C++ seperti yang dicantumkan pada bagian [Ingin Tahu Lebih](#).



Ayo Berlatih!

Aktivitas Individu

Aktivitas SAP-K11-09: Latihan Karakter dan String

1. Rancang dan buatlah sebuah program yang dapat membaca sebuah string dan mencetaknya secara terbalik.
2. Rancang dan buatlah sebuah program yang membaca sebuah kata sandi dan mencetak jumlah karakter yang berupa angka, huruf, dan simbol. Bedakan huruf kapital dan non kapital.

Ingin
Tahu Lebih?



1. Referensi terhadap fungsi-fungsi yang terdapat pada pustaka `string.h` dapat diakses di en.cppreference.com/w/cpp/header/cstring.
2. Referensi terhadap tipe data `string` pada bahasa pemrograman C++ dapat diakses di en.cppreference.com/w/cpp/string/basic_string.



Ayo Renungkan!

Setelah selesai melakukan aktivitas tersebut. Jawablah pertanyaan berikut ini dalam Lembar Refleksi pada Buku Kerja, dan jangan lupa mencatat kegiatan dalam Jurnal Peserta Didik.

1. Seberapa paham kalian dengan konsep larik pada pemrograman?
2. Seberapa paham kalian dengan konsep karakter dan string pada pemrograman?
3. Seperti apa pengalaman yang kalian rasakan saat membaca dokumentasi bahasa pemrograman yang disediakan?
4. Pelajaran paling berkesan yang kalian dapatkan dari aktivitas latihan ini?

D. Latihan Strategi Algoritmik dan Pemrograman Lintas Bidang

Pada bagian ini kalian akan membuat berbagai program berdasarkan permasalahan yang tersedia, tiap permasalahan memiliki sub permasalahannya tersendiri yang tingkat kesulitannya meningkat. Kalian akan membuat program dimulai dari perancangan, yaitu merancang algoritma untuk menyelesaikan permasalahan tersebut. Selanjutnya algoritma tersebut kalian terjemahkan ke dalam bahasa pemrograman yang kalian kuasai, misalnya bahasa C, Python, dsb. Permasalahan tersebut akan meningkatkan kemampuan programming kalian dengan mempelajari bagian ini dengan menyelesaikan berbagai sub permasalahan yang tersedia.

1. Problem Simulasi Burung

Pada bagian ini, kalian akan membuat program untuk mensimulasikan gerak burung yang diluncurkan dengan menggunakan alat ketapel. Secara prinsip, gerakan burung yang diluncurkan dengan menggunakan ketapel menggunakan **prinsip gerak lurus berubah beraturan (GLBB)**. Terdapat komponen sudut, gravitasi serta kecepatan dan waktu yang menjadi penentu jauhnya burung tersebut dapat meluncur dengan menggunakan ketapel. Konsep ini menggunakan kaidah gerak parabola yang telah kalian pelajari pada Mata

Pelajaran Fisika. *Problem* akan dibagi menjadi beberapa *subproblem* dengan tingkat kesulitan yang meningkat. Ikutilah petunjuk guru kalian dalam memilih tingkat kesulitan *subproblem* yang akan kalian kerjakan. Apabila kalian berhasil mengerjakan *subproblem* tersebut, kalian dapat menantang diri kalian untuk mengerjakan subproblem yang lebih sulit.

Setelah ditentukan tingkat kesulitan subproblem yang akan dikerjakan, kalian dapat mulai mengerjakan aktivitas Ayo Merancang Program: [Merancang Algoritma Simulasi Burung](#) dan Ayo Buat Program: [Membuat Program Simulasi Burung](#) berdasarkan deskripsi permasalahan yang sesuai.



 **Gambar 2.15**
Ilustrasi Boro yang sedang tidur

Problem: Program Simulasi Burung

Boro adalah seekor burung yang terjatuh dari sarangnya di sebuah pohon saat sedang tidur. Saat terjatuh, sayap Boro menghantam tanah dan ia sangat merasa kesakitan. Boro harus kembali ke sarangnya, namun karena sayapnya terluka, ia tidak dapat terbang sebagaimana mestinya. Tak jauh dari tempat Boro jatuh, terdapat ketapel raksasa yang biasa digunakan oleh pemilik lahan untuk kegiatan sirkus. Ketapel tersebut biasa digunakan untuk kegiatan menembak dengan menggunakan buah semangka.

Kebetulan saat itu, pemilik lahan sedang latihan sirkus menembak semangka dengan menggunakan ketapel. Boro meminta tolong kepada pemilik lahan, apakah ia dapat ikut di atas semangka tersebut agar dapat kembali ke rumahnya di atas pohon. Sang pemilik lahan setuju akan membantu dan menanyakan kepada Boro tentang lokasi pohon tempat tinggalnya.

Subproblem 1: Menghitung Jarak Horizontal Terjauh

Deskripsi:

Pada *subproblem* ini, kalian akan diminta untuk menghitung jarak horizontal terjauh yang dapat ditempuh oleh Boro apabila Boro ikut berdiri di atas semangka yang akan diluncurkan oleh pemilik lahan.

Format Masukan:

Baris pertama adalah sebuah bilangan bulat S yang menggambarkan sudut peluncuran. Nilai S ini bernilai $0-90$. Baris kedua adalah V yang merupakan kecepatan awal Boro saat meluncur dengan menggunakan ketapel. Asumsikan bahwa nilai gravitasi adalah 10

Format Keluaran:

Keluaran berupa bilangan yang menunjukkan jarak terjauh Boro mendarat di tanah.

▼ Tabel 2.3 Format Keluaran Program Simulasi Burung Subproblem 1

Contoh Masukan	Contoh Keluaran
37 10	9.6

Subproblem 2: Menghitung Waktu

Deskripsi:

Pada subproblem ini, kalian akan diminta untuk menghitung waktu yang diperlukan Boro untuk mencapai jarak horizontal terjauh apabila Boro ikut berdiri di atas semangka yang akan diluncurkan oleh pemilik lahan.

Format Masukan:

Baris pertama adalah sebuah bilangan bulat S yang menggambarkan sudut peluncuran. Nilai S ini bernilai $0-90$. Baris kedua adalah V yang merupakan kecepatan awal Boro saat meluncur dengan menggunakan ketapel. Asumsikan bahwa nilai gravitasi adalah 10

Format Keluaran:

Keluaran berupa bilangan yang menunjukkan waktu yang ditempuh Boro untuk mencapai jarak terjauh dengan format 3 angka di belakang koma.

▼ Tabel 2.4 Format Keluaran Program Simulasi Burung Subproblem 2

Contoh Masukan	Contoh Keluaran
37	12.036
100	

Subproblem 3: Prediksi Ketinggian Dicapai Boro

Deskripsi:

Pada *subproblem* ini, kalian akan diminta untuk memberikan prediksi apakah Boro dapat mencapai ketinggian lebih tinggi daripada tinggi pohon tempat ia bersarang, apabila Boro ikut berdiri di atas semangka yang akan diluncurkan oleh pemilik lahan.

Format Masukan:

Baris pertama adalah sebuah bilangan bulat S yang menggambarkan sudut peluncuran. Nilai S ini bernilai 0–90. Baris kedua adalah V yang merupakan kecepatan awal Boro saat meluncur dengan menggunakan ketapel. Baris ketiga adalah T yang merupakan tinggi pohon tempat Boro Bersarang. Asumsikan bahwa nilai gravitasi adalah 10.

Format Keluaran:

Keluaran berupa bilangan yang menunjukkan status ketinggian Boro dibandingkan dengan tinggi pohon tempat sarang Boro. 1 apabila Boro dapat mencapai ketinggian sama dengan tinggi pohon tempat sarangnya berada atau lebih, 0 apabila Boro tidak mampu mencapai ketinggian yang sama dengan pohon tersebut. Serta ketinggian maksimum yang dapat diperoleh oleh Boro saat meluncur dengan menggunakan ketapel.

▼ **Tabel 2.5** Format Keluaran Program Simulasi Burung Subproblem 3

Contoh Masukan	Contoh Keluaran
37 100 100	Status : 1 Ketinggian : 181.09
37 100 200	Status : 0 Ketinggian : 181.09

Subproblem 4: Prediksi Ketinggian Dicapai Boro dan Teman-temannya

Deskripsi:

Pada subproblem ini, kalian akan diminta untuk memberikan prediksi apakah Boro dan teman-temannya dapat mencapai ketinggian lebih tinggi daripada tinggi pohon tempat ia bersarang. Pada *subproblem* ini, burung yang akan menumpang semangka tidak hanya Boro, namun terdapat burung lain yang akan menumpang semangka tersebut secara bergantian.

Format Masukan:

Baris pertama adalah sebuah bilangan bulat N yang menggambarkan jumlah burung yang akan ikut peluncuran. Baris kedua adalah S yang menggambarkan sudut peluncuran. Nilai S ini bernilai $0-90$. Baris ketiga adalah V yang merupakan kecepatan awal Burung saat meluncur dengan menggunakan ketapel. Baris keempat adalah T yang merupakan tinggi pohon tempat burung bersarang. Asumsikan bahwa nilai gravitasi adalah 10 .

Format Keluaran:

Keluaran berupa urutan burung lalu bilangan yang menunjukkan status ketinggian burung dibandingkan dengan tinggi pohon tempat sarang burung tersebut. 1 apabila burung dapat mencapai ketinggian sama dengan

tinggi pohon tempat sarangnya berada atau lebih, 0 apabila burung tidak mampu mencapai ketinggian yang sama dengan pohon tersebut. Serta ketinggian maksimum yang dapat diperoleh oleh burung saat meluncur dengan menggunakan ketapel.

▼ **Tabel 2.6** Format Keluaran Program Simulasi Burung Subproblem 4

Contoh Masukan	Contoh Keluaran
1	Status Burung 1 : 1
37	Ketinggian: 181.09
100	Status Burung 1 : 1
100	Ketinggian: 181.09
2	Status Burung 2 : 0
37	Ketinggian: 181.09
100	
100	
37	
100	
200	



Ayo Merancang Program!

Aktivitas Berpasangan

Aktivitas SAP-K11-10-U: Merancang Algoritma Simulasi Burung

Berdasarkan deskripsi permasalahan di atas, secara individu, definisikanlah permasalahan dan rancanglah algoritma solusi dari permasalahan tersebut. Kalian dapat membuka kembali bahan belajaryangterkaittentangsimulasi burungyangmenjadi domain permasalahan yang diberikan. Dokumentasikanlah setiap langkah yang kalian kerjakan, termasuk apa yang kalian hasilkan dalam Buku Kerja kalian.

Setelah kalian selesai merancang algoritma, secara berpasangan, saling tukarkan algoritma kalian. Setelah

itu, telusurilah algoritma teman kalian dan cek apakah algoritma tersebut sudah benar atau belum. Apabila belum benar, secara bersama-sama, diskusikanlah apa yang dapat diperbaiki dari rancangan algoritma kalian. Jangan lupa untuk membandingkan solusi yang telah kalian hasilkan. Apabila solusi kalian berbeda, tapi sama-sama menghasilkan jawaban yang benar, bandingkanlah kedua solusi tersebut.



Ayo Buat Program!

Aktivitas Berpasangan

Aktivitas SAP-K11-11: Membuat Program Simulasi Burung

Sekarang, secara individu, implementasikanlah algoritma yang telah kalian rancang dalam bentuk program dengan menggunakan bahasa pemrograman yang telah kalian kuasai. Sebelum program kalian kompilasi, secara berpasangan, saling tukarkan kode program kalian dan cek apakah kode program tersebut sudah ditulis dengan benar. Setelah itu, kompilasi kode tersebut menjadi program, dan ujlilah program teman kalian dengan kasus uji yang kalian rancang. Apabila program teman kalian belum menghasilkan jawaban yang benar, sampaikanlah kepada teman kalian agar ia dapat memperbaiki kode program tersebut hingga menghasilkan jawaban yang benar. Setelah selesai, presentasikanlah hasil kerja kalian di depan kelas, mengikuti petunjuk dari guru.



Ayo Renungkan!

Setelah selesai melakukan aktivitas tersebut. Jawablah pertanyaan berikut ini dalam Lembar Refleksi pada Buku Kerja, dan jangan lupa mencatat kegiatan dalam Jurnal Peserta Didik.

1. Apa yang kalian rasakan saat membuat suatu algoritma dan program dari suatu permasalahan di mata pelajaran lain yang telah kalian ikuti?
2. Apakah program yang kalian buat dapat membantu kalian atau orang lain?
3. Apakah solusi yang telah kalian buat dapat dimanfaatkan untuk menyelesaikan masalah lain yang sejenis?
4. Adakah pengembangan lebih lanjut (enhancing) yang terpikir oleh kalian agar program menjadi lebih bermanfaat?
5. Apa yang kalian rasakan saat memeriksa solusi algoritma dan program teman kalian?
6. Apa yang kalian rasakan saat solusi algoritma dan program kalian diperiksa oleh teman kalian?
7. Pelajaran paling berkesan apa yang kalian dapatkan dari latihan ini?

2. Problem Pengelolaan Bank Darah

Padabagianini,kalianakanmenyelesaikansuatu permasalahan yang terkait dengan golongan darah. Informasi mengenai golongan darah sangat berguna di bidang kesehatan, bahkan dapat menyelamatkan nyawa seseorang. Dalam problem ini, kalian akan membantu sebuah rumah sakit untuk memastikan apakah stok darah yang mereka miliki cukup.

Problem: Pengelolaan Bank Darah

Suatu Bank Darah di Rumah Sakit mengelola stok darah yang mereka miliki. Persediaan darah yang mereka miliki dikelompokkan berdasarkan golongan darah standar A, AB, B, dan O seperti yang selama ini kita kenal. Kondisi stok darah yang aman adalah tersedianya cadangan yang cukup untuk setiap golongan darah. Apabila terdapat golongan darah yang stoknya menipis atau habis, maka

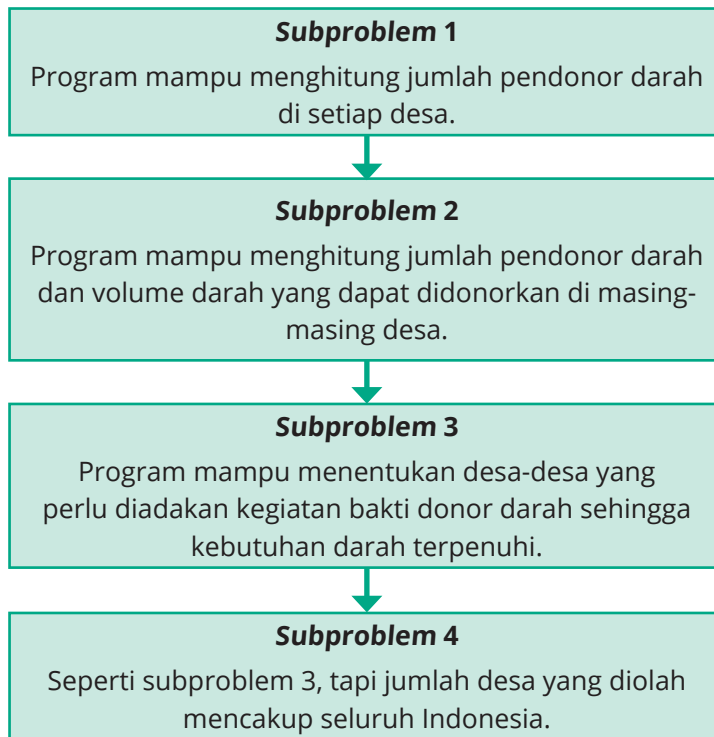
pihak Bank Darah akan melakukan kegiatan bakti donor darah di suatu desa. Masalah ini di dunia nyata lebih kompleks karena mempertimbangkan rhesus darah dan faktor lainnya, namun kita sederhanakan dengan hanya mempertimbangkan golongan darah pendonor.

Agar kegiatan bakti donor darah dapat berjalan secara efektif dan tepat sasaran, pihak Bank Darah mencatat data para Donor Darah Sukarela (DDS) yang tersebar di berbagai daerah. Informasi mengenai DDS yang dicatat oleh pihak Bank Darah adalah informasi kode desa, golongan darah, dan volume darah yang dapat didonorkan oleh DDS. Contoh data tersebut adalah sebagai berikut:

▼ Tabel 2.7 Contoh Data DDS Program Pengelolaan Bank Darah

Kode DDS	1	2	3	4	5	6	7	8
Kode Desa	3	1	3	1	2	3	1	2
Golongan Darah	A	B	A	AB	B	O	O	A
Volume Darah yang Dapat Didonorkan (ml)	150	250	300	450	200	350	500	500

Karena semakin lama, data DDS yang harus dikelola oleh Bank Darah tersebut semakin banyak, bahkan bisa jutaan orang, mereka meminta kalian untuk membangun sebuah program yang dapat menyimpan dan mengolah data DDS tersebut. Hasil pengolahan data tersebut akan digunakan oleh Bank Darah untuk menentukan pelaksanaan kegiatan Bakti Donor Darah yang lebih efektif dan tepat sasaran. Program tersebut harus bisa menentukan desa-desa yang harus dikunjungi untuk diadakan kegiatan bakti donor darah. Agar kalian merasakan proses pemrograman yang iteratif, kalian akan membuat program tersebut melalui empat tahap berikut:



▲ Gambar 2.16 Empat Tahap Pembuatan Program Pengelolaan Bank Darah

Catatan: Untuk menghindari kesulitan dalam entri data golongan darah AB, *problem* ini akan memberikan kode golongan darah A = 1, B = 2, AB = 3, dan O = 4. Kalian boleh saja mengkode dengan kode lainnya dengan syarat harus konsisten.

Subproblem 1: Menghitung Jumlah Pendonor Darah

Deskripsi:

Pada *subproblem* ini, kalian akan diminta untuk menghitung banyaknya pendonor darah di setiap desa berdasarkan kode desa yang diberikan.

Format Masukan:

Baris pertama adalah sebuah bilangan bulat N yang merupakan jumlah DDS. Nilai N ini paling banyak berjumlah 1000 orang. Baris kedua adalah M yang merupakan jumlah desa yang paling banyak berjumlah

10 desa. N baris berikutnya masing-masing akan terdiri atas tiga buah informasi, yaitu kode desa, golongan darah, dan volume darah yang dapat didonorkan. Kode desa merupakan sebuah bilangan bulat dari 1 sampai M, sedangkan volume darah yang dapat didonorkan adalah sebuah bilangan bulat dari 0 hingga 500 ml.

Format Keluaran:

Keluaran berupa banyaknya pendonor yang ada di masing-masing desa. Jumlah pendonor dicetak dengan format Desa M: Banyaknya Pendonor.

▼ Tabel 2.8 Format Keluaran Program Pengelolaan Bank Darah Subproblem 1

Contoh Masukan	Contoh Keluaran
8	Desa 1: 3
3	Desa 2: 2
3 1 150	Desa 3: 3
1 2 250	
3 1 300	
1 3 450	
2 2 200	
3 4 350	
1 4 500	
2 1 500	

Subproblem 2: Menghitung Jumlah Pendonor dan Total Volume Darah

Deskripsi:

Pada *subproblem* ini, kalian akan diminta untuk menghitung jumlah pendonor darah dan total volume darah yang dapat didonorkan di setiap desa berdasarkan kode desa dan golongan darah.

Format Masukan:

Sama dengan *subproblem* 1.

Format Keluaran:

Keluaran berupa jumlah pendonor dan total volume darah yang ada di masing-masing desa berdasarkan golongan darahnya. Ikuti format yang ada di contoh keluaran berikut:

▼ **Tabel 2.9** Format Keluaran Program Pengelolaan Bank Darah Subproblem 2

Contoh Masukan	Contoh Keluaran
8	Desa 1:
3	A: 0 0
3 1 150	B: 1 250
1 2 250	AB: 1 450
3 1 300	O: 1 500
1 3 450	Desa 2:
2 2 200	A: 1 500
3 4 350	B: 1 200
1 4 500	AB: 0 0
2 1 500	O: 0 0
	Desa 3:
	A: 2 450
	B: 0 0
	AB: 0 0
	O: 1 350

Subproblem 3: Memilih Tempat Pelaksanaan Donor Darah

Deskripsi:

Pada *subproblem* ini, kalian akan diminta membantu Bank Darah dalam memilih tempat pelaksanaan Bakti Donor Darah ketika stok suatu golongan darah habis. Karena pelaksanaan bakti donor darah membutuhkan biaya, kalian harus meminimumkan jumlah desa dimana bakti donor darah akan dilaksanakan, dengan tetap mencapai jumlah volume darah yang dibutuhkan.

Format Masukan:

Sama dengan *subproblem* 1. Setelah seluruh data DDS dimasukkan, baris berikutnya terdiri atas dua bilangan yang merupakan kode golongan darah yang habis, dan volume darah yang dibutuhkan (dalam ml). Nilai volume darah ini paling banyak bernilai satu juta mililiter.

Format Keluaran:

Apabila kebutuhan darah dapat dipenuhi, cetak daftar kode-kode desa yang akan dilaksanakan bakti golongan darah beserta jumlah darah yang dibutuhkan, dan satu kalimat “Kebutuhan darah dipenuhi dengan surplus X ml”. Sebaliknya, apabila kebutuhan darah tidak dapat dipenuhi, cetak “Kebutuhan darah masih kurang X ml” setelah seluruh keluaran dicetak.

Misalnya pada contoh kasus uji di *subproblem* 2, jika ditambahkan dengan 2 300 (artinya, ada kebutuhan 300ml golongan darah B) maka program haruslah menghasilkan keluaran:

```
Desa 1 : 250 ml  
Desa 2 : 200 ml  
Kebutuhan darah dipenuhi dengan surplus 150 ml
```

Karena dengan mengadakan bakti donor darah di kedua desa (Desa 1 dan Desa 2), kita dapat mengumpulkan total 450ml darah golongan B, sehingga menyisakan kelebihan 150ml. Tidak ada cara lain yang dapat dilakukan untuk memenuhi kebutuhan darah golongan B dengan mengadakan bakti donor darah di satu desa saja.

Subproblem 4: Tingkatkan Penyimpanan dan Kecepatan Olah Data DDS

Deskripsi:

Subproblem 4 sama dengan Subproblem 3. Hanya saja yang berbeda adalah ukuran problem yang diberikan. Program kalian sekarang harus bisa menyimpan dan

mengolah data para Donor Darah Sukarela dari seluruh Indonesia, sehingga batasan masukan naik menjadi:

Jumlah DDS : maksimal 30 juta orang
Jumlah Desa : maksimal 100000 desa

Selain itu, karena pengambilan keputusan harus diambil secara cepat, program kalian harus dapat berjalan dalam waktu paling lama 1 detik. Buatlah program yang dapat melakukan hal tersebut.



Ayo Merancang Program!

Aktivitas Berpasangan

Aktivitas SAP-K11-12-U: Merancang Algoritma Pengelolaan Donor Darah

Berdasarkan deskripsi permasalahan di atas, secara individu, definisikanlah permasalahan dan rancanglah algoritma solusi dari permasalahan tersebut. Kalian dapat membuka kembali bahan belajar yang terkait tentang pengelolaan donor darah yang menjadi domain permasalahan yang diberikan. Dokumentasikanlah setiap langkah yang kalian kerjakan, termasuk apa yang kalian hasilkan dalam Buku Kerja kalian.

Setelah kalian selesai merancang algoritma, secara berpasangan, saling tukarkan algoritma kalian. Setelah itu, telusurilah algoritma teman kalian dan cek apakah algoritma tersebut sudah benar atau belum. Apabila belum benar, secara bersama-sama, diskusikanlah apa yang dapat diperbaiki dari rancangan algoritma kalian. Jangan lupa untuk membandingkan solusi yang telah kalian hasilkan. Apabila solusi kalian berbeda, tapi sama-sama menghasilkan jawaban yang benar, bandingkanlah kedua solusi tersebut.



Ayo Buat Program!

Aktivitas Berpasangan

Aktivitas SAP-K11-13: Membuat Program Pengelolaan Donor Darah

Sekarang, secara individu, implementasikanlah algoritma yang telah kalian rancang dalam bentuk program dengan menggunakan bahasa pemrograman yang telah kalian kuasai. Sebelum program kalian kompilasi, secara berpasangan, saling tukarkan kode program kalian dan cek apakah kode program tersebut sudah ditulis dengan benar. Setelah itu, kompilasi kode tersebut menjadi program, dan ujilah program teman kalian dengan kasus uji yang kalian rancang. Apabila program teman kalian belum menghasilkan jawaban yang benar, sampaikanlah kepada teman kalian agar ia dapat memperbaiki kode program tersebut hingga menghasilkan jawaban yang benar. Setelah selesai, presentasikanlah hasil kerja kalian di depan kelas, mengikuti petunjuk dari guru.

► Ini Berpikir Komputasional!

Kegiatan yang telah kalian kerjakan di atas adalah menerapkan pemrograman untuk menyelesaikan permasalahan yang ada di dunia nyata. Terlihat bahwa informatika dapat digunakan untuk mengambil keputusan yang lebih baik berdasarkan data akurat yang diolah dengan cepat. Tentunya, problem yang diberikan di atas adalah penyederhanaan dari problem yang ada di lapangan, yang memiliki kompleksitas yang lebih tinggi.

Pada saat membaca deskripsi subproblem keempat, kalian mungkin kaget melihat jumlah data yang sangat besar. Program yang telah kalian buat untuk mengerjakan Subproblem 3 belum tentu dapat mengerjakan Subproblem 4 dalam waktu maksimal 1 detik yang diberikan. Yang berbeda dari Subproblem 3 dengan Subproblem 4 adalah ukuran dari problem yang lebih besar. Di sinilah salah satu tantangan dari big data, yaitu bagaimana kita dapat merancang algoritma yang paling efisien sehingga data yang besar tadi tetap dapat diolah dalam waktu yang singkat.



Ayo Renungkan!

Setelah selesai melakukan aktivitas tersebut. Jawablah pertanyaan berikut ini dalam Lembar Refleksi pada Buku Kerja, dan jangan lupa mencatat kegiatan dalam Jurnal Peserta Didik.

1. Apa yang kalian rasakan saat membuat algoritma dan program untuk membantu suatu lembaga?
2. Apakah program yang kalian buat dapat membantu kalian atau orang lain?
3. Apakah solusi yang telah kalian buat dapat dimanfaatkan untuk menyelesaikan masalah lain yang sejenis?
4. Adakah pengembangan lebih lanjut (enhancing) yang terpikir oleh kalian agar program menjadi lebih bermanfaat?
5. Apa yang kalian rasakan saat memeriksa solusi algoritma dan program teman kalian?
6. Apa yang kalian rasakan saat solusi algoritma dan program kalian diperiksa oleh teman kalian?
7. Prinsip Berpikir Komputasional apa saja yang kalian terapkan saat menyelesaikan aktivitas di atas?
8. Pelajaran paling berkesan apa yang kalian dapatkan dari latihan ini?

3. Problem Persilangan Tanaman

Pada bagian ini, kalian akan membuat program untuk mensimulasikan persilangan tanaman menggunakan **hukum Mendel** yang telah kalian pelajari pada Mata Pelajaran Biologi. *Problem* akan dibagi menjadi beberapa *subproblem* dengan tingkat kesulitan yang meningkat. Ikutilah petunjuk guru kalian dalam memilih tingkat kesulitan *subproblem* yang akan kalian kerjakan. Apabila kalian berhasil mengerjakan *subproblem*

tersebut, kalian dapat menantang diri kalian untuk mengerjakan *subproblem* yang lebih sulit.

Setelah ditentukan tingkat kesulitan *subproblem* yang diberikan, kalian dapat mengerjakan aktivitas Ayo Merancang Program: **Merancang Algoritma Persilangan Tanaman** dan Ayo Buat Program: **Membuat Program Persilangan Tanaman** berdasarkan deskripsi permasalahan yang sesuai.

Problem: Persilangan Benih Padi Unggulan

Tania, seorang peneliti di bidang pemuliaan tanaman, sedang melakukan persilangan tanaman padi untuk menghasilkan benih unggulan. Sifat tanaman padi yang akan ia perhatikan adalah tinggi tanaman, jumlah anakan, jumlah bulir, dan ketahanan terhadap penyakit. Seluruh sifat ini adalah dominan, dengan kata lain gen tanaman yang tinggi mendominasi tanaman yang rendah. Untuk membantunya melakukan persilangan, Tania membuat sebuah program yang dapat membantunya menghitung rasio peluang munculnya tanaman dengan karakteristik tertentu apabila kedua “orang tua”-nya diketahui sifatnya.

Karena Tania baru bisa membuat program yang membaca bilangan, Tania harus membuat kode sendiri agar ia dapat memasukkan karakteristik tanaman padi yang akan ia silangkan. Ia membuat aturan pemberian kode seperti berikut:

Pertama, Tania memberikan kode 0 sampai 2 untuk mewakili genotipe dari masing-masing fenotipe. Misalnya pada fenotipe Tinggi Tanaman, Tania memberikan nilai 2 untuk genotipe TT, nilai 1 untuk genotipe Tt, dan nilai 0 untuk genotipe tt.

▼ Tabel 2.10 Kode Sifat Fenotipe dan Genotipe Tanaman Tania

Sifat	Fenotipe	Kode	Sifat	Fenotipe	Kode
Tinggi Tanaman	Tinggi (TT)	2	Jumlah Bulir	Banyak (BB)	2
	Tinggi (Tt)	1		Banyak (Bb)	1
	Pendek (tt)	0		Sedikit (bb)	0
Jumlah Anakan	B a n y a k (AA)	2	Ketahanan terhadap Penyakit	Resisten (RR)	2
	B a n y a k (Aa)	1		Resisten (Rr)	1
	Sedikit (aa)	0		Rentan (rr)	0

Kedua, Tania akan menuliskan masukan program berupa satu atau lebih nilai 0, 1, atau 2 yang merupakan sifat dari tanaman yang akan Ia silangkan. Angka di posisi pertama merupakan tinggi tanaman, posisi kedua merupakan jumlah anakan, posisi ketiga merupakan jumlah bulir, dan posisi keempat adalah ketahanan terhadap penyakit. Sebagai contoh, jika Tania akan memasukkan tanaman yang tinggi (TT), jumlah anakan sedikit (aa), jumlah bulir sedikit (bb), dan resisten terhadap penyakit (Rr), maka bilangan yang akan Tania masukkan adalah: 2 0 0 1.

Sekarang, kalian dapat mulai membantu Tania membuat program. Selamat mengerjakan!

Subproblem 1: Menghitung Persentase Suatu Sifat Keturunan Pertama (F1)

Deskripsi:

Pada subproblem ini, Tania hanya memperhatikan sifat Tinggi Tanaman dan Jumlah Anakan, dan ingin menghitung persentase dari suatu keturunan generasi pertama (F1).

Format Masukan:

Masukan berupa tiga baris. Dua baris pertama adalah sifat dari kedua orang tua, dan baris ketiga adalah sifat dari keturunan pertama yang akan dihitung persentase kemunculan fenotipenya.

Format Keluaran:

Keluaran berupa sebuah persentase fenotipe dari tanaman di baris ketiga yang ditulis sebagai bilangan pecahan dengan dua tempat desimal dan diakhiri dengan tanda %.

▼ Tabel 2.11 Format Keluaran Program Persilangan Tanaman Subproblem 1

Contoh Masukan	Contoh Keluaran
2 2 0 0 0 0	25.00%

Penjelasan:

Pada contoh tersebut, Tania menyilangkan dua induk, yang bersifat tinggi (TT) dan banyak anakan (AA) dengan pendek (tt) dan sedikit anakan (aa). Lalu, Tania ingin menghitung persentase generasi pertama dengan sifat pendek (tt) dan sedikit anakan (aa). Rasio fenotipe dari persilangan ini adalah Tinggi dan Banyak Anakan : Pendek dan Sedikit Anakan = 3 : 1 = 75% : 25%.

Subproblem 2: Mengetahui Semua Sifat Keturunan Pertama (F1)

Deskripsi:

Pada subproblem ini, Tania hanya memperhatikan sifat Tinggi Tanaman dan Jumlah Anakan, dan ingin mengetahui sifat dari semua keturunan pertama (F1).

Format Masukan:

Masukan berupa dua baris yang berisi sifat dari kedua orang tua.

Format Keluaran:

Keluaran berupa 16 baris, masing masing baris berisi sifat dari semua keturunan pertama.

▼ **Tabel 2.12** Format Keluaran Program Persilangan Tanaman Subproblem 2

Contoh Masukan	Contoh Keluaran
2 2	Tinggi Banyak
1 1	Tinggi Banyak
	Tinggi Banyak
	Tinggi Banyak
	Tinggi Banyak
	Tinggi Sedikit
	Tinggi Banyak
	Tinggi Sedikit
	Tinggi Banyak
	Tinggi Banyak
	Pendek Banyak
	Pendek Banyak
	Tinggi Banyak
	Tinggi Sedikit
	Pendek Banyak
	Pendek Sedikit

Penjelasan:

Pada contoh tersebut, Tania menyilangkan dua induk, yang bersifat tinggi (TT) dan banyak anakan (AA) dengan pendek (tt) dan sedikit anakan (aa). Lalu, Tania ingin menghitung persentase generasi kedua dengan sifat pendek (tt) dan sedikit anakan (aa). Rasio fenotipe dari persilangan ini adalah Tinggi dan Banyak Anakan : Pendek dan Sedikit Anakan = 15 : 1 = 93.75% : 6.25%.

Subproblem 3: Menghitung Persentase Suatu Sifat Keturunan Kedua (F2)

Deskripsi:

Pada *subproblem* ini, Tania akan memperhatikan keempat sifat tanaman padi tersebut, dan ingin menghitung persentase dari suatu keturunan generasi kedua (F2).

Format Masukan:

Masukan berupa tiga baris. Dua baris pertama adalah sifat dari kedua orang tua, dan baris ketiga adalah sifat dari keturunan kedua yang akan dihitung persentase kemunculan fenotipenya.

Format Keluaran:

Keluaran berupa sebuah persentase fenotipe dari tanaman di baris ketiga yang ditulis sebagai bilangan pecahan dengan dua tempat desimal dan diakhiri dengan tanda %.

▼ Tabel 2.13 Format Keluaran Program Persilangan Tanaman Subproblem 3

Contoh Masukan	Contoh Keluaran
2 2 2 2 0 0 0 0 0 0 0 0	0.39%

Penjelasan:

Pada contoh tersebut, Tania menyilangkan dua induk, yang bersifat:

- 1: Tinggi (TT), Banyak Anakan (AA), Banyak Bulir (BB), dan Resisten (RR)
- 2: Pendek (tt), Sedikit Anakan (aa), Sedikit Bulir (bb), dan Rentan (rr)

Lalu, Tania ingin menghitung persentase generasi kedua dengan sifat:

Pendek (tt), Sedikit Anakan (aa), Sedikit Bulir (bb), dan Rentan (rr). Persentase fenotipe generasi kedua dari sifat tersebut adalah sama dengan 0.390625% yang jika dibulatkan ke atas menjadi 0.39%.



Ayo Merancang Program!

Aktivitas Berpasangan

Aktivitas SAP-K11-14-U: Merancang Algoritma Persilangan Tanaman

Berdasarkan deskripsi permasalahan di atas, secara individu, definisikanlah permasalahan dan rancanglah algoritma solusi dari permasalahan tersebut. Kalian dapat membuka kembali bahan belajar yang terkait tentang persilangan tanaman yang menjadi domain permasalahan yang diberikan. Dokumentasikanlah setiap langkah yang kalian kerjakan, termasuk apa yang kalian hasilkan dalam Buku Kerja kalian.

Setelah kalian selesai merancang algoritma, secara berpasangan, saling tukarkan algoritma kalian. Setelah itu, telusurilah algoritma teman kalian dan cek apakah algoritma tersebut sudah benar atau belum. Apabila belum benar, secara bersama-sama, diskusikanlah apa yang dapat diperbaiki dari rancangan algoritma kalian. Jangan lupa untuk membandingkan solusi yang telah kalian hasilkan. Apabila solusi kalian berbeda, tapi sama-sama menghasilkan jawaban yang benar, bandingkanlah kedua solusi tersebut.



Ayo Buat Program!

Aktivitas Berpasangan

Aktivitas SAP-K11-15: Membuat Program Persilangan Tanaman

Sekarang, secara individu, implementasikanlah algoritma yang telah kalian rancang dalam bentuk program dengan

menggunakan bahasa pemrograman yang telah kalian kuasai. Sebelum program kalian kompilasi, secara berpasangan, saling tukarkan kode program kalian dan cek apakah kode program tersebut sudah ditulis dengan benar. Setelah itu, kompilasi kode tersebut menjadi program, dan ujilah program teman kalian dengan kasus uji yang kalian rancang. Apabila program teman kalian belum menghasilkan jawaban yang benar, sampaikanlah kepada teman kalian agar ia dapat memperbaiki kode program tersebut hingga menghasilkan jawaban yang benar. Setelah selesai, presentasikanlah hasil kerja kalian di depan kelas, mengikuti petunjuk dari guru.

► Ini Berpikir Komputasional!

Pada Kelas X, kalian telah belajar bahwa Informatika dapat diterapkan di berbagai bidang keilmuan. Aktivitas di atas adalah salah satu contoh penerapan Informatika di bidang Biologi. Pada saat mengerjakan aktivitas di atas, kalian memanfaatkan pengetahuan kalian di pemrograman dan juga biologi untuk menyelesaikan permasalahan yang diberikan. Pada aktivitas ini juga kalian merasakan pengalaman bekerja berpasangan dalam membuat program. Membuat program yang kompleks seringkali melibatkan suatu tim yang harus bekerja sama dan saling melengkapi satu sama lain. Yang kalian lakukan di aktivitas ini adalah bagaimana kalian sebagai tim dapat saling mengecek hasil pekerjaan untuk menghasilkan program yang benar dan berkualitas tinggi. Pada aktivitas ini juga, kalian telah merasakan bagaimana Berpikir Komputasional diterapkan dalam pemrograman. Kalian dapat mencoba mengidentifikasi prinsip Berpikir Komputasional apa saja yang kalian terapkan dalam menyelesaikan problem tersebut.



Ayo Renungkan!

Setelah selesai melakukan aktivitas tersebut. Jawablah pertanyaan berikut ini dalam Lembar Refleksi pada Buku Kerja , dan jangan lupa mencatat kegiatan dalam Jurnal Peserta Didik.

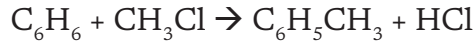
1. Apa yang kalian rasakan saat membuat suatu algoritma dan program dari suatu permasalahan di mata pelajaran lain yang telah kalian ikuti?
2. Apakah program yang kalian buat dapat membantu kalian atau orang lain?
3. Apakah solusi yang telah kalian buat dapat dimanfaatkan untuk menyelesaikan masalah lain yang sejenis?
4. Adakah pengembangan lebih lanjut (enhancing) yang terpikir oleh kalian agar program menjadi lebih bermanfaat?
5. Apa yang kalian rasakan saat memeriksa solusi algoritma dan program teman kalian?
6. Apa yang kalian rasakan saat solusi algoritma dan program kalian diperiksa oleh teman kalian?
7. Prinsip Berpikir Komputasional apa saja yang kalian terapkan saat menyelesaikan aktivitas di atas?
8. Pelajaran paling berkesan apa yang kalian dapatkan dari latihan ini?

4. Problem Stoikiometri

Stoikiometri adalah ilmu yang mempelajari dan menghitung hubungan kuantitatif dari reaktan dan produk dalam reaksi kimia. Pada bagian ini kalian akan diajak untuk membangun sebuah program yang dapat membantu kalian untuk dapat memproses masukan berupa suatu persamaan reaksi kimia. Manusia melakukan observasi pada reaksi kimia pada alam, dan dengan menggunakan berbagai hukum seperti hukum kekekalan massa, hukum perbandingan tetap, dan hukum perbandingan berganda, melahirkan suatu model reaksi kimia. Salah satunya adalah **stoikiometri**.

Setelah memahami stoikiometri, kalian dapat membaca sebuah persamaan reaksi kimia dan mengetahui bahwa persamaan tersebut benar karena semua pereaksi dikonsumsi

(habis bereaksi), tidak ada kekurangan pereaksi, dan tidak ada kelebihan pereaksi (sisa). Sebelum masuk ke *problem* yang akan dikerjakan, kalian dapat mencoba menuliskan secara struktur proses yang kalian lakukan untuk mengecek persamaan reaksi berikut apakah benar atau tidak.

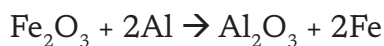


Problem: "Simulasi Stoikiometri"

Tujuan kalian dalam *problem* ini adalah membuat program yang dapat mengecek benar tidaknya persamaan reaksi kimia yang diberikan. Program yang dapat mengecek seluruh kemungkinan persamaan kimia sangat kompleks. Oleh karena itu, kita memerlukan batasan-batasan yang dapat menyederhanakan masalah tersebut, yaitu:

- Penulisan unsur kimia pada masukan sudah sesuai dengan aturan. Misal: Li untuk litium dan H untuk hidrogen.
- Persamaan reaksi yang diproses tidak melibatkan senyawa ionik dan senyawa yang memiliki tanda kurung.
- Persamaan reaksi yang diproses dijamin benar.

Selain itu, aturan penulisan yang menggunakan IUPAC seperti pada persamaan reaksi kimia berikut akan disederhanakan. Sebagai contoh, suatu persamaan reaksi kimia berikut



akan kita tulis dalam *problem* ini sebagai berikut:

1 Fe 2 O 3
2 Al 1
>
1 Al 2 O 3
1 Fe 2

Jika kalian tertarik, kalian bisa saja membuat program yang dapat membaca masukan berupa persamaan reaksi kimia seperti yang biasa dituliskan dengan mempelajari konsep parsing. **Parsing** adalah sebuah proses menganalisis *string* yang mengikuti aturan formal tertentu sehingga *string* tersebut dapat dibaca oleh program. Dalam hal ini, aturan formal yang diikuti adalah aturan formal penulisan reaksi kimia. Parsing tidak dibahas pada buku ini, namun kalian dapat mencari tahu lebih lanjut tentang konsep parsing dari berbagai sumber lainnya.

Subproblem 1: Menghitung Atom

Deskripsi:

Pada *subproblem* ini, kalian harus membuat program untuk menghitung banyaknya atom unsur tertentu yang terdapat pada senyawa kimia yang diberikan. Program kalian akan membaca sebuah senyawa dengan format yang telah ditentukan di bagian sebelumnya, kemudian mencetak banyaknya atom dari masing-masing unsur penyusun senyawa tersebut.

Format Masukan:

Sebuah senyawa yang telah diformat sesuai dengan deskripsi *problem*.

Format Keluaran:

Untuk setiap unsur, cetak lambang dari unsur tersebut dan banyaknya atom yang menyusun unsur tersebut. Kedua nilai ini dipisahkan oleh satu karakter spasi dan diakhiri dengan *newline*. Lambang unsur dan banyaknya atom dicetak berurutan mulai dari unsur paling kiri pada penulisan senyawa.

▼ Tabel 2.14 Format Keluaran Program Stoikiometri Subproblem 1

Contoh Masukan	Contoh Keluaran
1 C 1 H 3 Cl 1	C 1 H 3 Cl 1
2 H 2 O 1	H 4 O 2

Penjelasan:

Senyawa pada contoh pertama adalah CH_3Cl dan contoh kedua adalah $2\text{H}_2\text{O}$.

Subproblem 2: Mengecek Persamaan Reaksi

Deskripsi:

Pada *subproblem* ini, kalian akan membuat program yang akan membaca sebuah persamaan reaksi kimia, dan menentukan apakah persamaan reaksi kimia tersebut setimbang. Suatu atom dijamin hanya akan muncul 1 kali pada reaktan dan 1 kali pada produk. Apabila setimbang, maka program cukup mengeluarkan pesan “Reaksi Setimbang”. Apabila persamaan tersebut tidak setimbang, maka program akan mengeluarkan pesan “Reaksi Tidak Setimbang” dan menyajikan banyak atom dari reaktan yang kurang atau berlebih.

Format Masukan:

Sebuah persamaan reaksi kimia dengan format yang telah dijelaskan pada bagian penjelasan *problem*. Untuk mengakhiri masukan, kalian akan menginput sebuah karakter “F”.

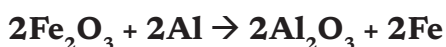
Format Keluaran:

Jika persamaan setimbang, cetak “Reaksi Setimbang”. Jika reaksi tidak setimbang, program akan mencetak lambang dari unsur yang kurang atau berlebih beserta

dengan selisih jumlah atomnya. Unsur dicetak berurutan mulai dari unsur yang paling kiri. Untuk setiap unsur tersebut, cetak dalam satu baris dengan format Lambang Unsur: jumlah atom. Jumlah atom diberi tanda negatif jika kurang, dan diberi tanda positif jika berlebih.

Contoh:

Misalnya program menerima masukan berupa persamaan reaksi kimia berikut:



Program akan melihat bahwa persamaan tersebut tidak setimbang dan akan mengeluarkan keluaran berikut:

▼ Tabel 2.15 Format Keluaran Program Stoikiometri Subproblem 2

Contoh Masukan	Contoh Keluaran
2 Fe 2 O 3 2 Al 1 > 2 Al 2 O 3 2 Fe 1 F	Reaksi Tidak Setimbang Fe:+2 Al:-2

Subproblem 3: Menghitung Hasil Reaksi

Deskripsi:

Pada *subproblem* ini, program akan menerima masukan berupa persamaan reaksi kimia. Bagian reaktan dari persamaan tersebut sudah ditulis dengan koefisien yang lengkap. Namun, koefisien dari bagian produk persamaan tersebut belum lengkap. Suatu atom dijamin hanya akan muncul 1 kali pada reaktan dan 1 kali pada produk. Setelah membaca persamaan tersebut, program akan mencetak koefisien dari setiap molekul di bagian produk.

Format Masukan:

Sebuah persamaan reaksi kimia dengan format yang telah dijelaskan pada bagian penjelasan *problem*, akan

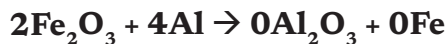
tetapi koefisien pada seluruh molekul produk diberi nilai 0. Untuk mengakhiri masukan, kalian akan menginput sebuah karakter “F”.

Format Keluaran:

Keluaran dari program adalah koefisien dari setiap molekul produk yang dihasilkan dari persamaan tersebut. Koefisien tersebut harus membuat persamaan reaksi yang setimbang. Jika bagian produk terdiri atas lebih dari satu molekul, koefisien dicetak dari molekul produk yang paling kiri dan dipisahkan dengan karakter spasi.

Contoh:

Misalnya program menerima masukan berupa persamaan reaksi kimia berikut:



Program akan melihat bahwa persamaan tersebut tidak setimbang dan akan mengeluarkan keluaran berikut:

▼ Tabel 2.16 Format Keluaran Program Stoikiometri Subproblem 3

Contoh Masukan	Contoh Keluaran
2 Fe 2 O 3 4 Al 1 > 0 Al 2 O 3 0 Fe 1 F	2 4

Dengan koefisien 2 dan 4 pada sisi produk, maka persamaan reaksi kimia akan setimbang menjadi



Subproblem 4: Melengkapi Koefisien Reaktan dan Produk

Deskripsi:

Pada *subproblem* ini, persamaan reaksi kimia yang dibaca seluruhnya memiliki koefisien 0. Program harus membaca persamaan reaksi kimia tersebut dan mencetak koefisien setiap molekul sedemikian sehingga reaksi setimbang. Karena kemungkinan koefisien ini sangat banyak, kalian hanya perlu mencetak kemungkinan koefisien yang paling kecil. Terdapat tepat 2 senyawa pada reaktan, dan 2 senyawa pada hasil dan masing-masing atom hanya muncul tepat 1 kali pada reaktan dan 1 kali pada hasil.

Format Masukan:

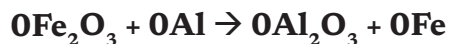
Sebuah persamaan reaksi kimia dengan format yang telah dijelaskan pada bagian penjelasan problem, akan tetapi koefisien pada seluruh molekul reaktan dan produk diberi nilai 0.

Format Keluaran:

Keluaran program adalah persamaan reaksi kimia yang telah dilengkapi dengan koefisien terkecil yang membuat persamaan reaksi tersebut setimbang.

Contoh:

Misalnya program menerima masukan berupa persamaan reaksi kimia berikut:



Program akan mencetak persamaan reaksi.

▼ Tabel 2.17 Format Keluaran Program Stoikiometri Subproblem 4

Contoh Masukan
1 Fe 2 O 3
2 Al 1
>
1 Al 2 O 3
2 Fe 1



Ayo Merancang Program!

Aktivitas Berpasangan

Aktivitas SAP-K11-16-U: Merancang Algoritma Simulasi Stoikiometri

Berdasarkan deskripsi permasalahan di atas, secara individu, definisikanlah permasalahan dan rancanglah algoritma solusi dari permasalahan tersebut. Kalian dapat membuka kembali bahan belajar yang terkait tentang stoikiometri yang menjadi domain permasalahan yang diberikan. Dokumentasikanlah setiap langkah yang kalian kerjakan, termasuk apa yang kalian hasilkan dalam **Buku Kerja** kalian.

Setelah kalian selesai merancang algoritma, secara berpasangan, saling tukarkan algoritma kalian. Setelah itu, telusurilah algoritma teman kalian dan cek apakah algoritma tersebut sudah benar atau belum. Apabila belum benar, secara bersama-sama, diskusikanlah apa yang dapat diperbaiki dari rancangan algoritma kalian. Jangan lupa untuk membandingkan solusi yang telah kalian hasilkan. Apabila solusi kalian berbeda, tapi sama-sama menghasilkan jawaban yang benar, bandingkanlah kedua solusi tersebut.



Ayo Buat Program!

Aktivitas Berpasangan

Aktivitas SAP-K11-17: Membuat Program Simulasi Stoikiometri

Sekarang, secara individu, implementasikanlah algoritma yang telah kalian rancang dalam bentuk program dengan menggunakan bahasa pemrograman yang telah kalian kuasai. Sebelum program kalian kompilasi, secara berpasangan, saling tukarkan kode program kalian dan cek apakah kode program tersebut sudah ditulis dengan benar. Setelah itu, kompilasi kode tersebut menjadi program, dan ujilah program teman kalian dengan kasus uji yang kalian rancang. Apabila program teman kalian belum menghasilkan jawaban yang benar, sampaikanlah kepada teman kalian agar ia dapat memperbaiki kode program tersebut hingga menghasilkan jawaban yang benar. Setelah selesai, presentasikanlah hasil kerja kalian di depan kelas, mengikuti petunjuk dari guru.

► Ini Berpikir Komputasional!

Pada problem stoikiometri ini, selain kalian membuat program yang mensimulasikan suatu aturan di bidang Kimia, kalian pun belajar membuat program yang menerima masukan yang dekat dengan bahasa yang digunakan manusia.



Ayo, Kita Renungkan

Setelah selesai melakukan aktivitas tersebut. Jawablah pertanyaan berikut ini dalam Lembar Refleksi pada Buku Kerja, dan jangan lupa mencatat kegiatan dalam Jurnal Peserta Didik.

1. Apa yang kalian rasakan saat membuat suatu algoritma dan program dari suatu permasalahan di mata pelajaran lain yang telah kalian ikuti?

2. Apakah program yang kalian buat dapat membantu kalian atau orang lain?
3. Apakah solusi yang telah kalian buat dapat dimanfaatkan untuk menyelesaikan masalah lain yang sejenis?
4. Adakah pengembangan lebih lanjut (*enhancing*) yang terpikir oleh kalian agar program menjadi lebih bermanfaat?
5. Apa yang kalian rasakan saat memeriksa solusi algoritma dan program teman kalian?
6. Apa yang kalian rasakan saat solusi algoritma dan program kalian diperiksa oleh teman kalian?
7. Prinsip Berpikir Komputasional apa saja yang kalian terapkan saat menyelesaikan aktivitas di atas?
8. Pelajaran paling berkesan apa yang Kalian dapatkan dari latihan ini?

E. Proyek Praktik Lintas Bidang (PLB)

Pertama-tama, sesuai dengan petunjuk dari guru, buatlah kelompok yang masing-masing terdiri dari 2 orang peserta didik (jika kelas berjumlah ganjil, satu kelompok boleh berisi 3 orang peserta didik). Saat mengerjakan proyek ini, diperlukan kerja sama antar anggota kelompok dengan baik untuk menghasilkan jawaban dan solusi yang tepat.

Tujuan utama dari proyek lintas bidang ini adalah menghasilkan sebuah program (dalam Bahasa C/C++) yang dapat menyelesaikan permasalahan *knapsack* (baik yang berjenis *rational knapsack* maupun *0-1 knapsack*) dengan menerapkan prinsip-prinsip penyelesaian masalah yang telah dipelajari pada bagian Berpikir Komputasional. Hasil akhir yang diharapkan adalah berupa dua hal:

- Laporan analisis dan perancangan program
- Kode program yang dapat dijalankan dan memberikan solusi dari setiap masukan yang diberikan

Dalam pengerjaan ini, kalian akan dipandu dengan serangkaian langkah-langkah. Ikutilah langkah-langkah tersebut dan kerjakan bersama rekan satu tim untuk mendapatkan hasil terbaik.

1. Penjelasan Permasalahan

Permasalahan yang akan diselesaikan pada PLB ini adalah salah contoh dari sebuah permasalahan optimasi. Dalam dunia informatika, optimasi adalah suatu tipe permasalahan dimana kita ingin mencari nilai yang optimal (paling baik) untuk sebuah kasus permasalahan tertentu. Konsep “paling baik” dapat berarti dua hal: paling kecil (minimal) atau paling besar (maksimal), sehingga secara umum ada dua jenis permasalahan optimasi, yaitu optimasi untuk mencari nilai maksimum dan optimasi untuk mencari nilai minimum. Nilai atau besaran yang ingin dioptimalkan (minimal atau maksimal) biasa disebut sebagai fungsi tujuan (fungsi obyektif).

Pada umumnya, ketika mencari suatu nilai yang optimal dalam sebuah permasalahan optimasi, kita harus menghadapi suatu kendala yang mengatur pilihan-pilihan yang dapat kita ambil. Mungkin, tidak semua pilihan dapat kita ambil. Ketika kita melakukan optimasi, kita harus memilih di antara pilihan-pilihan tersebut mana yang memberikan nilai fungsi obyektif yang paling optimal.

Sebagai contoh, kita tinjau kembali permasalahan penukaran uang yang telah kita pelajari pada bagian [Berpikir Komputasional](#). Pada permasalahan tersebut, kita diberikan beberapa pecahan uang (misalnya: 100 Rupiah, 200 Rupiah, 500 Rupiah, 1000 Rupiah, 5000 Rupiah dan 10.000 Rupiah), serta sebuah besaran nilai uang yang ingin kita tukarkan, (misalnya Rp 27.800). Kita diminta untuk menentukan, berapa banyaknya pecahan uang terkecil yang dapat kita tukarkan untuk mencapai besaran tersebut?

Permasalahan ini adalah sebuah permasalahan optimasi dimana jenis optimasi pada permasalahan ini adalah pencarian

nilai minimum (terkecil). Fungsi tujuannya adalah banyaknya pecahan uang yang harus dikumpulkan, sedangkan kendala yang diberikan adalah besaran-besaran pecahan uang yang tersedia, serta jumlah total besaran nilai uang yang harus dikumpulkan.

Pada PLB ini kita akan mempelajari bagaimana menyelesaikan permasalahan yang biasa disebut sebagai knapsack problem. Berikut ini adalah deskripsi umum permasalahan knapsack problem.

Terdapat beberapa buah barang yang dapat kita ambil. Setiap barang memiliki dua hal: **bobot** dan **nilai**. Kita diberikan sebuah wadah yang dapat menampung beberapa buah barang, namun kapasitasnya berat beban yang dapat ditampungnya terbatas sampai suatu nilai tertentu (tidak boleh mengisi wadah melebihi batas maksimal kapasitas beban ini, namun tentunya boleh kurang). Pertanyaannya adalah: “Dapatkah kita memilih barang-barang yang diambil untuk dimasukkan ke dalam wadah, sedemikian rupa sehingga kita mendapatkan total nilai yang sebesar-besarnya, namun tidak melewati batas maksimal beban wadah tersebut?”

Sebagai contoh, perhatikanlah kasus berikut ini. Misalnya kita memiliki 5 buah barang, sebut saja namanya A, B, C, D dan E, dengan bobot (dalam kg) dan nilai masing-masing sebagai berikut:

▼ **Tabel 2.18** Keterangan Bobot dan Nilai dari 5 Barang (Knapsack Problem) Kasus 1

Barang	A	B	C	D	E
Bobot	5	4	7	8	10
Nilai	10	5	7	12	8

Misalkan kapasitas maksimal dari wadah adalah 20 kg. Maka kita dapat melakukan beberapa contoh pilihan sebagai berikut:

- Ambil barang A, B dan D. Total bobot yang harus ditampung pada wadah adalah $5 + 4 + 8 = 17$ kg, yang

masih di bawah maksimal kapasitas wadah. Total nilai yang kita dapatkan adalah $10 + 5 + 12 = 27$.

- Ambil barang B, C dan D, dengan total bobot = $4 + 7 + 8 = 19$ kg, dengan total nilai yang didapatkan = $5 + 7 + 12 = 24$.
- Ambil barang A, C dan D, dengan total bobot = $5 + 7 + 8 = 20$ kg, dengan total nilai yang didapatkan = $10 + 7 + 12 = 29$.

Tentunya kita tidak dapat mengambil A, B, C dan D sekaligus misalnya, karena total bobotnya adalah $5 + 4 + 7 + 8 = 24 > 20$ kg, sehingga akan melewati kapasitas dari wadah. Ternyata, nilai total 29 yang didapatkan dari memilih A, C dan D di atas adalah nilai maksimal yang kita dapatkan, dan tidak ada pemilihan pengambilan barang yang lain yang menghasilkan nilai total > 29 dengan tetap memenuhi persyaratan total bobot $<$ kapasitas.

2. Rational dan 0-1 Knapsack

Terdapat dua jenis variasi dari permasalahan *knapsack*. Yang pertama adalah yang disebut sebagai *rational knapsack*. Pada permasalahan ini, setiap barang dapat dianggap sebagai barang dapat dipecah, artinya, boleh diambil sebagian saja (tidak harus semuanya). Contoh barang seperti ini adalah misalnya: air, minyak, beras, pasir, dan lain-lain. Jika Anda memiliki 10 kg minyak, Anda dapat memilih untuk mengambil hanya 7 kg dari minyak tersebut, sehingga nilai yang Anda dapatkan adalah $7/10$ dari nilai total keseluruhan minyak. Karena setiap barang sifatnya dapat dipecah dan diambil sebagian, maka permasalahan *rational knapsack* biasanya lebih mudah diselesaikan, karena kita dapat mengatur untuk setiap barang, berapa bagian yang ingin kita ambil.

Variasi kedua dari permasalahan *knapsack* adalah yang disebut sebagai *0-1 knapsack*. Sesuai dengan namanya, pada permasalahan *0-1 knapsack*, kita hanya dapat memilih untuk mengambil atau tidak mengambil setiap barang, karena setiap barang sifatnya adalah tunggal dan tidak dapat

dipecah belah. Contoh barang seperti ini adalah misalnya: laptop, sepeda, mobil, hewan dan sebagainya, yang tentunya tidak dapat diambil hanya sebagian/sepotong saja, dan kita harus menentukan apakah setiap barang akan diambil atau tidak. Permasalahan 0-1 *knapsack* biasanya lebih sulit untuk diselesaikan daripada *rational knapsack*.

Perhatikan contoh kasus berikut ini. Diberikan tiga buah barang yang tidak dapat dibagi/dipecah, A, B dan C dengan bobot dan nilai sebagai berikut yang akan ditaruh dalam sebuah wadah.

▼ Tabel 2.19 Keterangan Bobot dan Nilai dari 3 Barang (Knapsack Problem) Kasus 2

Barang	A	B	C
Bobot	5	4	7
Nilai	10	5	7

Misalkan bahwa kapasitas wadah adalah 10 kg. Apabila kita mengasumsikan bahwa jenis permasalahan ini adalah *rational knapsack*, maka kita dapat mengambil solusi sebagai berikut:

- Ambil semua barang A, sehingga didapatkan bobot = 5 kg, total nilai sementara = 10.
- Kemudian ambil semua barang B, sehingga didapatkan bobot = $5 + 4 = 9$ kg, dan nilai total sementara = $10 + 5 = 15$.
- Ambil $1/7$ bagian dari barang C, sehingga didapatkan total bobot = $9 + 1/7 * 7 = 10$ kg, dan nilai total menjadi $15 + 1/7 * 7 = 16$.

Hasil ini ternyata adalah hasil yang maksimal, kita tidak dapat memperoleh nilai total yang lebih besar dari 16 tanpa melebihi kapasitas wadah, yaitu 10 kg.

Nah sekarang, seandainya kita menganggap permasalahan ini sebagai permasalahan 0-1 *knapsack*, maka kita hanya dapat mengambil setiap barang secara keseluruhan, atau tidak sama sekali. Maka dalam kasus ini, solusi optimalnya adalah

dengan mengambil barang A dan B saja, dengan total bobot = $5 + 4 = 9 < 10$ kg, dan total nilai = $10 + 5 = 15$.



Aktivitas PLB

Aktivitas Berpasangan

Aktivitas SAP-K11-18-U: Memahami Permasalahan Knapsack

Jawablah pertanyaan-pertanyaan berikut dan tuliskan dalam laporan analisis.

1. Apakah jenis optimasi pada permasalahan *knapsack*? Apakah mencari minimum, ataukah maksimum? Jelaskan!
2. Tentukan apa yang menjadi fungsi tujuan dari permasalahan *knapsack*!
3. Tentukan apa yang menjadi kendala pada optimasi untuk permasalahan *knapsack*!
4. Perhatikan permasalahan knapsack yang ditunjukkan oleh Tabel 2.20 berikut. Diberikan 6 buah barang, A, B, s/d F dengan bobot dan nilai sebagai berikut:

▼ Tabel 2.20 Keterangan Bobot dan Nilai dari 6 Barang (Knapsack Problem) Aktivitas PLB

Barang	A	B	C	D	E	F
Bobot	3	8	5	4	10	8
Nilai	6	4	5	6	5	10

Asumsikan bahwa tas memiliki kapasitas maksimal = 24 kg.

- a. Apakah pilihan mengambil barang-barang B, D, E dan F diperbolehkan sebagai solusi sesuai dengan kendala optimasi pada permasalahan tersebut? Mengapa?
- b. Apakah pilihan mengambil barang-barang A, D, E diperbolehkan sebagai solusi sesuai dengan kendala optimasi pada permasalahan tersebut? Apakah fungsi tujuan mencapai nilai optimal dengan memilih A, D dan E saja? Mengapa?

5. Tentukan jawaban permasalahan knapsack tersebut pada soal no. 4, jika menggunakan variasi permasalahan rational *knapsack*!
6. Pada soal no. 4, apakah solusinya, jika digunakan variasi 0-1 knapsack? Apakah sama dengan solusi untuk variasi rational *knapsack*?



Aktivitas PLB

Aktivitas Berpasangan

Aktivitas SAP-K11-19: Mengkodekan Permasalahan *Knapsack*

Membuat Skema Masukan Program

Sebelum kita dapat menyelesaikan permasalahan di atas dengan menggunakan sebuah program, hal pertama yang perlu kita lakukan adalah menentukan bagaimana menyatakan sebuah kasus permasalahan *knapsack* menjadi sebuah bentuk/format yang mudah dibaca dan diolah oleh komputer. Cara yang paling mudah adalah dengan menuliskan besaran-besaran yang ada pada permasalahan tersebut dan menyatakannya dalam *array-array* atau deretan bilangan yang sesuai. Kita dapat memulai dengan menuliskan banyaknya barang yang tersedia, kemudian diikuti dengan bobot masing-masing barang, kemudian diikuti dengan nilai masing-masing barang, dan terakhir kita tuliskan kapasitas maksimal dari tas. Sebagai contoh, untuk kasus *knapsack* dimana barang-barang yang akan dimasukkan pada tas ditunjukkan pada [Tabel 2.21](#) berikut:

▼ **Tabel 2.21** Keterangan Bobot dan Nilai dari 5 Barang (Knapsack Problem) Aktivitas PLB

Barang	A	B	C	D	E
Bobot	5	4	7	8	10
Nilai	10	5	7	12	8

Dan dimana jumlah maksimal bobot yang dapat ditampung adalah 20 kg, maka kita dapat merepresentasikan permasalahan ini menjadi bentuk sebagai berikut:

Contoh masukan 1

5

5 4 7 8 10

10 5 7 12 8

20

Sekarang perhatikan permasalahan pada [Tabel 2.22](#) berikut:

▼ **Tabel 2.22** Keterangan Bobot dan Nilai dari 6 Barang (Knapsack Problem) Aktivitas PLB

Barang	A	B	C	D	E	F
Bobot	3	8	5	4	10	8
Nilai	6	4	5	6	5	10

Dimana kapasitas maksimal yang dapat ditampung adalah 25 kg. Sekarang perhatikan juga skema masukan pada contoh berikut:

Contoh masukan 2

8

3 10 6 7 9 10 7 5

1 10 8 1 7 8 9 18

35

Jawablah pertanyaan-pertanyaan berikut pada lembar jawaban/laporan PLB!

Pertanyaan:

1. Tentukan pengkodean yang sesuai untuk permasalahan yang ditunjukkan pada Tabel 2.22 di atas, dengan menggunakan skema pengkodean yang telah dijelaskan. Jelaskan!
2. Tuliskan/jelaskan pada laporan analisis kalian, deskripsi permasalahan untuk contoh masukan 2 di atas!
3. Jelaskan dalam satu paragraf, skema pengkodean di atas, agar dapat dipahami oleh orang lain!

4. Mungkinkah sebuah representasi data tidak valid/tidak sesuai? Berikan contohnya, dan tuliskan penjelasannya pada laporan analisis!

Membaca Masukan dan Menyimpan Data

Sekarang, setelah kita memiliki sebuah representasi data yang sesuai untuk setiap permasalahan, marilah kita memulai untuk membuat program yang dapat menyelesaikan permasalahan yang telah dinyatakan dengan skema pengkodean tersebut. Pada bagian ini, kita akan membuat program yang membaca masukan dengan format di atas, kemudian hanya menampilkan kembali deskripsi soal sebagai keluaran. Perhatikan contoh di bawah ini.

Misalkan diberikan masukan berupa data sebagaimana ditunjukkan pada Contoh Masukan 1. Maka program harus mengeluarkan output sebagai berikut:

```
Banyak barang = 5
Barang 1: bobot = 5 kg, nilai = 10
Barang 2: bobot = 4 kg, nilai = 5
...
Barang 5: bobot = 10 kg, nilai = 8
Kapasitas maksimal = 20 kg
```

Untuk menyelesaikan permasalahan ini, kita perlu mengingat kembali konsep *array* pada Bagian Algoritma dan Pemrograman, dan menggunakannya untuk menyimpan data bobot dan nilai dari barang-barang yang tersedia.



Aktivitas PLB

Aktivitas Berpasangan

Aktivitas SAP-K11-20-U: Merancang Algoritma Penyelesaian Masalah Knapsack

Pada bagian **Berpikir Komputasional**, kita sudah mempelajari beberapa jenis teknik penyelesaian masalah. Untuk permasalahan

knapsack pada PLB ini, kita harus menentukan strategi penyelesaian masalah yang manakah yang sesuai untuk diterapkan.

1. Untuk permasalahan *rational knapsack*, tentukan apakah strategi *greedy* ataukah *dynamic programming* yang sesuai untuk diterapkan? Jelaskan pada laporan analisis kamu, bagaimana strategi *greedy* atau *dynamic programming* dapat diterapkan pada permasalahan *rational knapsack*!
2. Untuk permasalahan 0–1 *knapsack*, tentukan apakah strategi *greedy* ataukah *dynamic programming* yang sesuai untuk diterapkan?
3. Tuliskan dalam notasi *pseudocode* algoritma yang sesuai untuk menyelesaikan permasalahan *rational knapsack* menggunakan strategi yang Anda pilih pada bagian nomor 1!
4. **[Optional]** Tuliskan dalam notasi *pseudocode* algoritma yang sesuai untuk menyelesaikan permasalahan 0-1 *knapsack* menggunakan strategi yang Anda pilih pada bagian nomor 2!



Aktivitas PLB

Aktivitas Berpasangan

Aktivitas SAP-K11-21: Mengimplementasikan dan Menguji Program Solusi Knapsack

Implementasi program

Pada bagian ini, setiap kelompok akan membuat program yang menyelesaikan permasalahan *rational knapsack*. Program yang dibuat memiliki spesifikasi (ketentuan) sebagai berikut:

1. Program membaca masukan berupa representasi pengkodean kasus sebagaimana dijelaskan di atas.
2. Kemudian program tersebut menjalankan strategi penyelesaian program sebagaimana telah dirancang pada *pseudocode* pada bagian sebelumnya.
3. Program kemudian menghasilkan keluaran berupa sebuah angka yang menunjukkan berapa nilai total terbesar yang

dapat dicapai (nilai optimal) untuk kasus yang diberikan pada masukan.

Misalnya, jika program diberikan masukan sebagaimana contoh masukan 1, maka program akan memberikan keluaran sebagaimana dijelaskan pada [Contoh 1](#), yaitu 29.

Contoh Masukan	Contoh Keluaran
5 5 4 7 8 10 10 5 7 12 8 20	29

Untuk melakukan implementasi program ini, dianjurkan setiap kelompok melakukan sistem yang disebut sebagai *pair programming*. *Pair programming* adalah sebuah teknik pemrograman dimana dua orang bekerja bersama dalam menghasilkan sebuah program. Satu orang bertindak untuk mengerjakan penulisan program, satu orang lagi bertindak untuk mengawasi dan memberikan masukan terhadap penulisan program rekannya, sekaligus memikirkan strategi untuk penulisan program berikutnya. Selanjutnya kedua orang dapat bergantian peran, apabila diperlukan.

Jika kalian mengimplementasikan teknik *greedy* untuk menyelesaikan permasalahan *rational knapsack*, maka kalian akan memerlukan proses pengurutan data [[Ingat kembali materi Berpikir Komputasional di Kelas X!](#)]. Untuk melakukan hal ini, kalian dapat langsung saja menggunakan fitur dalam bahasa C++ yang menyediakan pustaka untuk melakukan pengurutan, yaitu dengan fungsi `sort()`. Jika kalian memiliki sebuah `std::vector<>` bernama data misalnya, maka kalian tinggal memanggil fungsi `sort()` sebagai berikut: `sort(data.begin(), data.end())`. Maka data pada *vector* tersebut akan terurut. Lebih lanjut, kita dapat menggunakan proses pengurutan tidak hanya pada satu data, tapi kita dapat juga menggunakan hasil pengurutan untuk mengurutkan

data yang lain. Perhatikan contoh berikut, dimana kita ingin menampilkan data nama-nama peserta didik sesuai dengan urutan nilai ujiannya, secara terurut menaik!

```
1:  /* Melakukan pengurutan pada dua buah vector */
2:  #include <stdio.h>
3:  #include <vector>
4:  #include <string>
5:  #include <algorithm>
6:
7:  int main()
8:  {
9:      std::vector<int> nilai {90, 60, 80, 75};
10:     std::vector<std::string>
11:         nama {"Andi", "Budi", "Cici", "Didi"};
12:
13:     std::vector<int> index;
14:     for(int i=0; i<4; i++) index.push_back(i);
15:
16:     // Urutkan index berdasarkan nilai
17:     std::sort(index.begin(), index.end(),
18:         [&nilai](int index1, int index2)
19:         { return nilai[index1] < nilai[index2]; });
20:
21:     // Cetak nama berdasarkan urutan nilai
22:     for(int i=0; i<4; i++)
23:         printf("%s ", nama[index[i]].c_str());
24:
25:     return 0;
26: }
```

Keluaran dari program tersebut adalah:

Budi Didi Cici Andi

Jika kita ingin mengurutkan secara menurun, kalian dapat merubah operasi perbandingan < pada baris berikut:

```
return nilai[index1] < nilai[index2];
```

Menjadi operasi perbandingan: >

```
return nilai[index1] > nilai[index2];
```

Untuk lebih jelasnya, dengarkan penjelasan gurumu mengenai hal ini!

Pengujian program

Setelah program diimplementasikan, langkah terakhir adalah melakukan pengujian. Pada tahap ini, setiap kelompok

membuat beberapa kasus uji. Setiap kasus uji berisi kedua komponen data, yaitu masukan dan keluaran yang telah diperiksa secara manual oleh kedua anggota kelompok, untuk memastikan kebenarannya. Setiap kelompok perlu membuat minimal 5 kasus uji yang panjangnya masing-masing berisi minimal 5 buah barang.

Selanjutnya, program diuji dengan kasus-kasus uji tersebut, dan diperiksa apakah keluaran dari program sesuai dengan keluaran yang diinginkan. Untuk menambah keketatan pengujian, setiap kelompok dapat “meminjam” kasus-kasus uji dari 2 kelompok lain untuk melakukan pengujian program masing-masing.

Tuliskan hasil pengujian keseluruhan pada laporan akhir PLB! Ikuti format berikut untuk melaporkan hasil pengujian.

▼ **Tabel 2.23** Format Hasil Pengujian Program Laporan Akhir PLB

Masukan	Keluaran yang diharapkan	Keluaran yang didapatkan	Hasil Pengujian
5 5 4 7 8 10 10 5 7 12 8 20	29	29	Sesuai
3 5 4 7 10 5 7	16	15	Tidak sesuai
....			



Ayo Renungkan!

Setelah selesai melakukan aktivitas tersebut. Jawablah pertanyaan berikut ini dalam Lembar Refleksi pada Buku Kerja, dan jangan lupa mencatat kegiatan dalam Jurnal Peserta Didik.

1. Apa yang kalian rasakan saat membuat suatu program di Praktik Lintas Bidang ini?
2. Apakah program yang kalian buat dapat membantu kalian atau orang lain?
3. Apakah solusi yang telah kalian buat dapat dimanfaatkan untuk menyelesaikan masalah lain yang sejenis?
4. Adakah pengembangan lebih lanjut (*enhancing*) yang terpikir oleh kalian agar program menjadi lebih bermanfaat?
5. Apa yang kalian rasakan saat memeriksa solusi algoritma dan program teman kalian?
6. Apa yang kalian rasakan saat solusi algoritma dan program kalian diperiksa oleh teman kalian?
7. Pelajaran paling berkesan apa yang kalian dapatkan dari aktivitas latihan ini?